MONASH University

New Interactive Visual Tools and Statistical Methodology for Selecting and Evaluating Non-linear Dimension Reduction Layouts of High-dimensional Data

Piyadi Gamage Jayani Lakshika
B.Sc. (Hons) in Statistics, University of Sri Jayewardenepura (USJ), Sri Lanka

A thesis submitted for the degree of
Doctor of Philosophy
at Monash University in 2025
Department of Econometrics & Business Statistics

Table of contents

Copyright notice

Abstract

High-dimensional datasets consist of observations characterized by numerous features, often exhibiting complex geometric properties that can be challenging to visualize. While linear dimension reductions are a reliable method for representing high-dimensional data, they can lead to cluttered visualizations where global patterns obscure local structures and can result in concentrating the data at the center of projections. To overcome these limitations, Non-linear dimension reduction (NLDR) techniques have been developed, applying non-linear transformations to provide clearer, low-dimensional representations of high-dimensional data. However, the effectiveness of NLDR methods can vary based on the choice of techniques and hyper-parameters, leading to diverse representations that can be either accurate or misleading. The main objective of this research is to develop new methods and software tools for understanding, and evaluating NLDR techniques to improve our understanding of high-dimensional data structures.

This research presents four original contributions. The first contribution (Chapter 2) introduces a new method for visualizing how NLDR warps data. This method improves the diagnostics of NLDR techniques. The second contribution (Chapter 3) provides evidence in identification of clusters at various distances when observing NLDR representation and the tour view of high-dimensional data. This finding is based on a human subject experiment that explores both the perception and misperception of NLDR representations. The third contribution (Chapter 4) presents two R packages: `quollr` and `cardinalR`. The `quollr` implements the method introduced in Chapter 2 as an R package. The `cardinalR` is developed to generate high-dimensional clustering data structures, with features such as adding noise dimensions and background noise. Finally, the fourth contribution (Chapter 5) features a Shiny app that offers a user-friendly interface for analysts to obtain the most accurate NLDR representation. Overall, this work advances the field of diagnosing NLDR by improving the visualization of high-dimensional data.

Declaration

I hereby declare that this thesis contains no material which has been accepted for the award of any other degree or diploma at any university or equivalent institution and that, to the best of my knowledge and belief, this thesis contains no material previously published or written by another person, except where due reference is made in the text of the thesis.

This thesis includes one original papers published in a peer reviewed journal and four unpublished papers. The core theme of the thesis is to "develop methods and software to understand non-linear dimension reduction methods". The ideas, development and writing up of all the papers in the thesis were the principal responsibility of myself, the student, working within the Department of Econometrics and Business Statistics under the supervision of Professor Dianne Cook, Dr Paul Harrison (MGBP, BDInstitute), Dr Michael Lydeamore, and Dr Thiyanga S. Talagala (University of Sri Jayewardenepura).

The inclusion of co-authors reflects the fact that the work came from active collaboration between researchers and acknowledges input into team-based research. In the case of Chapter 2, Chapter 4, and, Chapter 5, my contribution to the work involved the following:

Chapter	Publication title	Status	Student contribution	Co-authors contribution	Coauthors are Monash students
2		Revised and resubmitted in the Journal of Computational and Graphical Statistics	80% Concept, Analysis, Software, Writing		No

(continued)

Chapter	Publication title	Status	Student contribution	Co-authors contribution	Coauthors are Monash students
4		Submitted in the R Journal	80% Concept, Analysis, Software, Writing		No
4		Submitted in the R Journal	80% Concept, Analysis, Software, Writing		No
5		Submitted in the Oxford Academic (Bioinformatics Advances)	80% Concept, Analysis, Software, Writing		No

Chapters 3 is planned for submission to peer-reviewed journal.

To ensure the clarity and coherence of the written content, artificial intelligence tools were employed to assist in smoothing and refining the language throughout the thesis.

The thesis is written in Australian spelling, except for Chapters 2, 4, and 5, which use American spelling as specified by the publication venue.

I have renumbered sections of submitted papers in order to generate a consistent presentation within the thesis.

Student name: Piyadi Gamage Jayani Lakshika

Student signature:

Date: 13th January 2026

I hereby certify that the above declaration correctly reflects the nature and extent of the student's and co-authors' contributions to this work. In instances where I am not the responsible author I have consulted with the responsible author to agree on the respective contributions of the authors.

Main Supervisor name: Dianne Cook

Main Supervisor signature:

Date: 13th January 2026

Acknowledgements

This thesis would have never been a success without the guidance, encouragement, and support of my supervisors, family, and friends. This note is to show my gratitude to them.

First, I would like to sincerely thank my supervisors Professor Dianne Cook, Dr Paul Harrison, Dr Michael Lydeamore, and Dr Thiyanga S. Talagala for being incredible supervisors and mentors. Their endless enthusiasm for research, productivity, and wealth of knowledge has motivated me throughout my Ph.D. at Monash university. No matter how busy they were, they always kept the doors open to welcome me, whenever I needed their help. I believe that they are the best supervisors I could ever find in my life long research journey. Thank you for believing in my ability to navigate difficulties, even when I doubted myself. I am immensely grateful and honoured to have worked under guidance of you all. Your expertise, way of thinking, and leadership will continue to inspire me now and always.

I gratefully acknowledge the constructive feedback, suggestions, and encouragement I received from my PhD milestone evaluation panel members: Professor Catherine Forbes, Professor Xibin Zhang, Associate Professor Ruben Loaiza Maya, Dr Jessica Leung, Dr Shanika Wickramasuriya, and Dr Kate Saunders. Special thanks go to Professor Catherine Forbes, and Professor Xibin Zhang for being outstanding PhD directors and for their immense support throughout my candidature. I am also thankful to Professor Mervyn Silvapulle and Emeritus Professor Donald Poskitt, who taught our PhD coursework units during the first year of the programme.

I am thankful to Monash University, Australia, for this invaluable opportunity to pursue my PhD at such a prestigious institution. The exposure and experiences I gained over the years at Monash have been immense. I am particularly grateful for the financial support provided through the Co-funded Graduate Research Scholarship and the Monash Business School Co-Funded Graduate Research Scholarship, which enabled me to concentrate fully on my research. I am also thankful for the financial assistance provided to attend international conferences and visiting some invaluable mentors in the USA and Austria. I take this opportunity to express my sincere gratitude to all other academic and administrative staff members of the Department of Econometric and Business Statistics at Monash University.

I would like to sincerely thank Emeritus Professor Gael Martin, who, together with Professor Rob Hyndman, welcomed me into the department at the time of my application. I also thank Professor George Athanasopoulos for his generous assistance in making it possible for me to complete my visit to the USA. My heartfelt thanks go as well to Professor Brett Inder, Associate Professor Christo Karuna, and Professor Xueyan Zhao, whose warm smiles, encouragement, and constant support have been invaluable in helping me to complete this journey.

I would also like to thank Professor Heike Hofmann, Associate Professor Susan VanderPlas, Associate Professor Ursula Laa, Associate Professor Natalia da Silva, and Professor Eun-Kyung, whom I had the privilege of visiting during my PhD journey. Their generosity in providing opportunities, their guidance in improving my work, and their endless care and kindness have meant a great deal to me. I am also grateful to the University of Nebraska–Lincoln, USA and the University of Natural Resources and Life Sciences, Vienna, Austria, for warmly hosting me during these visits.

I acknowledge the use of Grammarly and ChatGPT for grammar and spelling checks, which helped improve the accuracy of my writing.

I remember with gratitude, the University of Sri Jayewardenepura, Sri Lanka, for paving the way for my academic journey. I am especially grateful to my undergraduate research supervisor, Dr Thiyanga S. Talagala, for sparking my interest in visualisation, and for broadening my academic horizons through her wisdom and mentorship. Without her guidance, I would never have thought to begin this journey, and I remain deeply grateful for that. I am also grateful to Sigithi Kindergarten, Weligama, Sri Sumangala Balika Vidyalaya, Weligama, and Sujatha Vidyalaya, Matara, Sri Lanka, where I received my early, primary, and secondary education. I extend my heartfelt thanks to all my teachers and private tutors, who continue to check on me even today. The lessons and challenges from those years made me stronger and more resilient, which proved invaluable during my PhD journey.

Chapter 2 is being revised for the **Journal of Computational and Graphical Statistics**. I take this opportunity to thank the two anonymous reviewers for their constructive comments. I presented my research work at 12^{th}-Conference of the Asian Regional Section of the International Association for Statistical Computing (IASC-ARS 2023) (Wollongon, Australia), Australian Statistical Conference (ASC 2023) (Wollongon, Australia), Bioinformatics Seminar 2024, Victorian branch of the Australian and New Zealand Industrial and Applied Mathematics Society (VicANZIAM) 2024 (RMIT university, Melbourne, Australia), Faculty of BusEco Three Minute Thesis (3MT) competition 2024, useR! 2024 (Salzburg, Austria), Graphics Group Presentation 2024 (Nebraska, USA), UNO Data Science Club 2024 (Omaha, USA), Joint Statistical Meetings (JSM) 2025 (Nashville, USA), useR! 2025 (Durham, USA), Biometrics in the Bush Capital (BIBC2025) (Canberra, Australia), and Australian Statistical

Conference (ASC 2025) (Perth, Western Australia). I would like to thank the participants of these seminars, research groups, and the conferences for their valuable comments.

I am deeply grateful to my extended family here in Melbourne Nuwani and Rehan, Heshani and Kanishka, and Shanika (also became my gym partner), who not only stood by me during difficult times and but constant listeners. My heartfelt thanks also go to Chaya and Supun, Himasha and Danushka, and Hiruni, for their genuine friendship and unwavering support.

I am thankful to my fellow PhD students at Monash for making this journey such a memorable experience. Special appreciation goes to my "Numbats family" Patrick, Mitch, Fin, Sherry, Cynthia, Harriet, Janith, Kris, David, and Hannah for the camaraderie and encouragement. I also treasure the many conversations with Nimni and Gayani during office hours, which brought lightness and motivation to my days. To my office mates: Shelly, Elvis, Floyd, Cash, Minh, and Vis thank you for creating a supportive and welcoming environment. I am especially grateful to Dr. Xiaoqian, both an office mate and a wonderful roommate during conference trips, for sharing this journey with me.

My sincere thanks go to Dovini and Vihanga for their help during my English test and the application process to Australia, and to the MIG family for their kindness and support throughout my PhD. I also extend my gratitude to Danusha, Kalani, Narmada, Malith, and all the friends I met during my visit to the University of Nebraska, USA, who made that experience so enriching. I will never forget the kindness of Dilmi and Nishadi, who welcomed me on my very first day in Melbourne, picked me up, and helped me settle into a new life. I remain truly grateful for their generosity. I am grateful to Pabasara, Vindula, Amaya, Nirmitha, and Malinda for being wonderful friends from my university days until now, and for continuing to support me through the difficult moments of this journey. I also sincerely thank Kavishka, Dinith, Sonal, Thilina, and Tharaka for their enduring friendship and encouragement throughout this time. A special thanks to Paton, Michael's dog, whose wagging tail and joyful spirit always lifted my mood.

Last, but by no means least, I would like to thank my family members. I am truly grateful to my parents for their unconditional selfless love, care, and sacrifices that they made to make my life better. Your inspiration and guidance always kept me motivated to perceive better achievements in my life. I am fortunate to be your daughter. I would also like to thank my sister, my brother, and my sister-in-law for their unconditional love, understanding, and moral support both in this academic journey and in life. No matter how I stumble, you have always lifted me up and reminded me of my worth. Amma, Thaththa, Nangi, Malli and Podi nangi I am endlessly grateful for you, beyond words, beyond life, and beyond everything.

This journey was never mine alone, and for that, I am eternally grateful!

Chapter 1

Introduction

High-dimensional data, where each observation is described by many variables, is increasingly prevalent in modern science from bioinformatics to computer vision. For example, **CITE-seq data** (**?**) simultaneously records RNA expression and protein markers for individual cells, producing rich, complex datasets. To interpret such data, analysts often rely on **dimension reduction** methods to create low-dimensional visualizations that can reveal patterns and structure.

One established approach is **linear projection**, where high-dimensional points are represented as linear combinations of the original features. **Principal Component Analysis (PCA)** (for an overview see Jolliffe (2011)) is the most familiar method, identifying directions of maximum variance. Extending this idea, **tours** (Lee et al. 2021) provide dynamic sequences of linear projections, giving views from multiple angles to help reveal hidden structure. Tour methods are implemented in R packages such as `tourr` (Wickham et al. 2011), `langevitour` (**?**), and `detourr` (Hart and Wang 2022). A key advantage of linear projections is that they preserve the geometric relationships of the original data, they do not introduce distortion. However, linear projections can become cluttered, and global structure may obscure local detail. Furthermore, **piling** (Laa et al. 2022) where points concentrate in the center of projections can mask important variation.

To overcome these limitations, analysts frequently turn to **nonlinear dimension reduction (NLDR)** methods such as tSNE (Maaten and Hinton 2008), UMAP (McInnes et al. 2018), PHATE (Moon et al. 2019), TriMAP (**?**), and PaCMAP (Wang et al. 2021). NLDR applies nonlinear transformations to generate low-dimensional embeddings that aim to preserve local or global data relationships. These methods are designed to **exaggerate structure**, making it easier for analysts to detect patterns that may not be apparent through linear projections.

Yet this strength also introduces a critical risk: **NLDR can hallucinate structure**, creating patterns

in the low-dimensional space that do not exist in the high-dimensional data. This issue is strikingly illustrated in **?@fig-NLDR-variety**, where eight different NLDR representations of the same CITE-seq dataset vary dramatically due to differences in method or hyperparameter choices. Such variability raises essential questions: *Which layout can be trusted? Which accurately reflects the high-dimensional data structure?*

Despite the widespread use of NLDR, there is no widely accepted or visually interpretable framework for **diagnosing the reliability of NLDR representations**. Analysts are left to rely on subjective judgment when choosing and interpreting NLDR layouts, without tools to distinguish faithful representations from artifacts. There is also a lack of benchmark clustering datasets with known geometric structure for testing NLDR methods systematically.

In addition to technical gaps, **little is known about how people perceive and misperceive structure in NLDR layouts**. It is unclear how different visualizations influence analysts' conclusions, or how distortions introduced by NLDR affect decision making. Given the critical role of visualization in high-dimensional data analysis, understanding the human perception of NLDR representations is essential.

1.1 Research Objectives

This thesis addresses these challenges through four main objectives:

1. **Develop methods and software to diagnose NLDR techniques**, providing tools to assess whether low-dimensional representations accurately reflect high-dimensional structures.
2. **Conduct a large-scale user study to explore perception and misperception in NLDR representations**, assessing how viewers recognize structure differently in NLDR layouts compared to tour-based views. This cognitive perception experiment will help identify common mistakes made when choosing and reporting structure from NLDR visualizations, and will inform best practices for using these methods in data analysis.
3. **Create benchmark datasets with known geometric properties**, via the `cardinalR` package, to systematically evaluate and validate NLDR methods.
4. **Provide a platform for NLDR evaluation**, combining linked linear projections (such as tours) and NLDR layouts to reveal where distortions arise and how structure is preserved or lost.

1.2 Contribution

By addressing both computational and cognitive aspects of NLDR, this research aims to advance our understanding of when and how NLDR methods can be trusted. The work will provide practical tools, benchmark data, and empirical insights to support the responsible use of dimension reduction techniques in high-dimensional data analysis.

1.3 Thesis Outline

The rest of the thesis is organized as follows:

Chapter 2 introduces an algorithm to assess the NLDR and decide on which, if any, is the most reasonable representation of the structure(s) present in high-dimensional data. We create a model starting with NLDR layout that is then used to display as a wireframe in high dimensions.

Chapter 3 provides empirical evidence on how viewers recognize structure differently when using NLDR layouts versus the tour view, particularly with varying distances between clusters. The findings will help clarify common mistakes made when selecting and reporting structures based on NLDR layouts.

Chapter 4 introduces two R packages developed as part of this research. Section 4.1 presents the implementation of the work is available as an R package, named `quollr`, an acronym to "**qu**estioning how a high-dimensional **o**bject **l**ooks in **l**ow-dimensions using **r**" (**?**). This package also contains a function for performing hexagonal binning using a new approach, for saving langevitour results with a specific projection, and link plots to understand the quirks that occur with different NLDR techniques. Section 4.2 proposes the R package, `cardinalR` (**?**) (**c**ollection of v**ar**ious high-**d**imens**i**o**nal** data structures in **R**), which includes functions to generate high-dimensional clustering data structures, with features such as adding noise dimensions and background noise along with some already generated examples.

Chapter 11 introduces `menuraR` (**m**onitoring **e**mbeddings of **n**onlinear **u**nfoldings for **r**epresentation and **a**nalysis in **R**), a Shiny web application designed to select and evaluate NLDR layouts.

Chapter 12 concludes the thesis, summarises the contribution of the work, and discusses some future plans.

Chapter 2

Visualising how non-linear dimension reduction warps your data

Chapter 3

A user study to explore perception and misperception in NLDR representations

Chapter 4

Software

4.1 quollr: An R Package for Visualising 2-D Models from Non-linear Dimension Reductions in High Dimensional Space

4.2 cardinalR: Generating interesting high-dimensional data structures

A high-dimensional dataset is one where each observation is described by many features, or dimensions, with associations between them. These datasets contain nonlinear manifolds in image and speech recognition, clusters in genomics and forensic analysis, and sparse distributions in text mining. Data with a variety of structures can be generated using mathematical functions and statistical distributions to create test datasets. High-dimensional data structures are useful for testing, validating, and improving algorithms used in dimensionality reduction, clustering, machine learning, and visualization. Their controlled complexity allows researchers to understand challenges posed in data analysis and helps to develop robust analytical methods across diverse scientific fields like bioinformatics, machine learning, and forensic science. Functions to generate a large variety of structures in high dimensions are organized into the R package `cardinalR`, along with some already generated examples, adding to the existing toolset of benchmark datasets for evaluating algorithms.

Chapter 5

Introduction

Generating synthetic datasets with clearly defined geometric properties is useful for evaluating and benchmarking algorithms in various fields, such as machine learning, data mining, and computational biology. Researchers often need to generate data with specific dimensions, noise characteristics, and complex underlying structures to test the performance and robustness of their methods. There are numerous packages available in R for generating synthetic data, each designed with unique characteristics and focus areas.The `geozoo` package ((**?**)) offers a large collection of geometric objects, allowing users to create and analyze specific shapes, primarily in lower-dimensional spaces. The package is `snedata` ((**?**)), which provides tools for generating simplified datasets useful for evaluating dimensionality reduction techniques like tSNE, often focusing on understanding and evaluating low-dimensional embeddings of complex data structures. Additionally, `splatter` ((**?**)) is designed to simulate complex biological data, capturing field-specific nuances such as batch effects and differential expression. In contrast, `mlbench` ((**?**)) includes a collection of well-known benchmark datasets commonly associated with established classification or regression challenges. The `surreal` package ((**?**)) implements the "Residual (Sur)Realism" algorithm ((**?**)) to generate datasets that embed hidden images or text into residual plots, providing engaging visual demonstrations for teaching model diagnostics. Meanwhile, the `DHARMa` package ((**?**)) adopts a simulation-based approach to create scaled quantile residuals for generalized linear (mixed) models and related frameworks, supporting model diagnostics through intuitive residuals, plots, and tests for common misspecification issues.

While these packages are valuable, their scope is often limited to specific applications or low-dimensional structures. To address this gap, this paper introduces the `cardinalR` R package. This package provides a collection of functions designed to generate customizable data structures in any number of dimensions, starting from basic geometric shapes. `cardinalR` offers important functionalities that extend beyond the capabilities of existing tools, allowing users to: (i) construct

high-dimensional datasets based on geometric shapes, including the option to enhance dimensionality by adding controlled noise dimensions; (ii) introduce adjustable levels of background noise to these structures; and (iii) combine high-dimensional datasets into a single multi-faceted, clustered dataset in a space of arbitrary dimension. By using clearly defined geometric shapes and controllable characteristics such as number of dimensions, sample size; `cardinalR` allows researchers to generate transparent and interpretable synthetic datasets useful for evaluating the performance of nonlinear dimensionality reduction (NLDR) methods, clustering algorithms, and visualization techniques. Moreover, these datasets can serve as benchmark examples for exploring how different algorithmic choices affect the identification or representation of cluster and manifold structures in high-dimensional spaces.

The paper is organized as follows. In the next section, we introduce the implementation of the `cardinalR` package on GitHub, including a demonstration of the package's key functions. We illustrate how a clustering data structure affects the dimension reductions in the Application section. Finally, we give a brief conclusion of the paper and discuss potential opportunities for the use of our data collection.

Chapter 6

Implementation

The `cardinalR` package is built on a modular framework where individual geometric generators (e.g., Gaussian, cone, sphere) create well-defined shapes. The main function, `gen_multicluster()`, combines these shapes into a single dataset by applying scaling, rotation, and translation through `gen_rotation()`. Each generated shape is assigned a unique cluster label. This design allows flexible construction of complex, high-dimensional structures for evaluating clustering and dimension reduction methods. Figure @ref(fig:workflow) illustrates the workflow of `gen_multicluster()`.

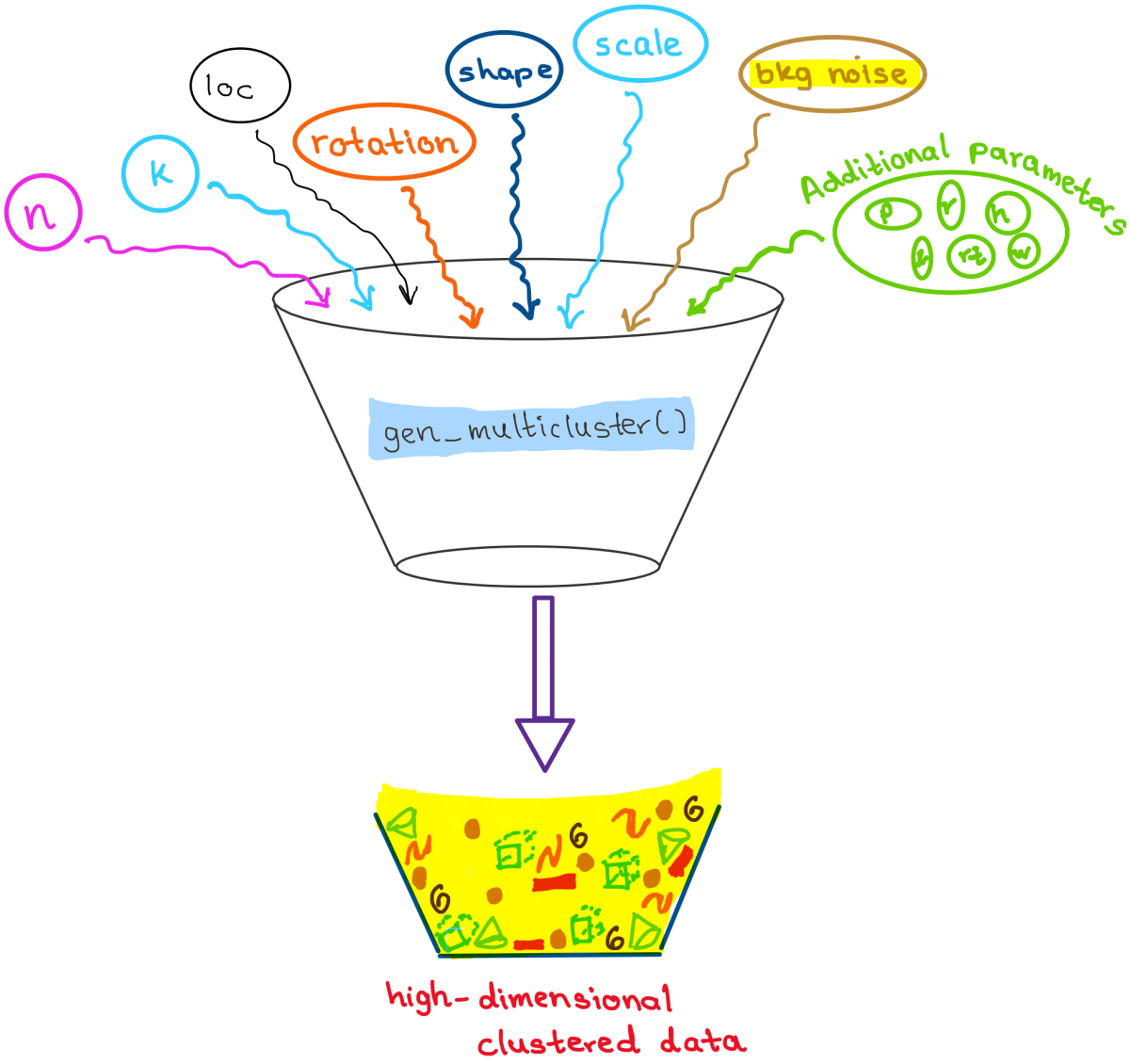

Figure 6.1: *Workflow for generating high-dimensional clustered data. The user specifies input parameters (number of points, clusters, cluster shapes, scaling, rotation, and optional background noise). Clusters are generated iteratively, transformed, optionally augmented with Gaussian noise dimensions, combined, and labeled, resulting in the final dataset.*

Chapter 7

Usage

The `cardinalR` R package is available on GitHub at JayaniLakshika/cardinalR.

7.1 Main function

The main function of the package is `gen_multicluster()`, which generates datasets consisting of multiple clusters with user-specified characteristics. Users can control the number of clusters (`k`), and the number of points in each cluster (`n`). Each cluster can take on a different geometric shape (e.g., Gaussian, cone, uniform cube) by specifying the corresponding generator function (`shape`), can be scaled to adjust its spread, rotated using custom rotation matrices (`rotation`), and positioned at defined centroids (`loc`). The function ensures flexibility in cluster location and orientation, allowing users to simulate complex high-dimensional structures.

To maintain consistency across generators, the function identifies the arguments required by each chosen generator function and supplies only those arguments that are valid for that specific generator. This design enables the combination of cluster types with differing parameter requirements within the same dataset. When clusters are generated with fewer dimensions than others, the function augments the lower-dimensional clusters with additional Gaussian noise variables so that all clusters are represented in the same dimensional space. These noise dimensions are drawn independently from normal distributions

$$X \sim \mathcal{N}(m, s^2),$$

where the mean (m) is set to the average of the cluster coordinates and the standard deviation (s) defaults to 0.2.

Table 7.1: *The main arguments for gen_multicluster().*

Argument	Type	Explanation
n	numeric (vector)	Number of points in each cluster.
k	numeric	Number of clusters.
loc	numeric (matrix)	Locations/centroids of clusters.
scale	numeric (vector)	Scaling factors of clusters.
shape	character (vector)	Shapes of clusters.
rotation	numeric (list)	Rotation matrices, one per cluster.
is_bkg	boolean	Background noise should exist or not.

An optional argument, `is_bkg`, adds background noise drawn from a multivariate normal distribution centered on the dataset's overall mean with standard deviations matching the observed spread. Extra arguments `(...)` can be passed to cluster generators, allowing further control over per-cluster characteristics like radius of the sphere. The main arguments of the `gen_multicluster()` function are shown in Table @ref(tab:main-tb-pdf).

The following example demonstrates how to use `gen_multicluster()` to create a 4-*D* dataset with three clusters of different shapes and orientations:

```
# Define example rotation matrices for 4D space
rot1 <- gen_rotation(p = 4, planes_angles = list(
  list(plane = c(1, 2), angle = 60),
  list(plane = c(3, 4), angle = 90)
))

rot2 <- gen_rotation(p = 4, planes_angles = list(
  list(plane = c(1, 3), angle = 30)
))

rot3 <- gen_rotation(p = 4, planes_angles = list(
  list(plane = c(2, 4), angle = 45)
))

# Generate the clustered dataset
clust_data <- gen_multicluster(
  n = c(200, 300, 500),
  k = 3,
```

```
  loc = matrix(c(
    0, 0, 0, 0,
    5, 9, 0, 0,
    3, 4, 10, 7
  ), nrow = 3, byrow = TRUE),
  scale = c(3, 1, 2),
  shape = c("gaussian", "cone", "unifcube"),
  rotation = list(rot1, rot2, rot3),
  is_bkg = FALSE
)
```

7.2 Shape generators

The shape generators form the foundation of the package, providing functions to create synthetic datasets from simple, well-defined geometric forms such as cones, pyramids, spheres, grids, and branching structures. Each generator includes the parameter `n`, which specifies the number of points to generate. Some functions, such as `gen_unifcube()`, also take the dimension `p`, while others include arguments specific to the geometry (e.g., radius for spheres (`r`), width for bands (`w`)). If higher-dimensional data are required, additional noise dimensions can be appended after data generation using any noise generator function. This flexibility allows users to construct both low- and high-dimensional datasets from the same underlying structures.

7.2.1 Branching

A branching structure (Figure @ref(fig:branch-proj)) captures trajectories that diverge or bifurcate from a common origin, similar to processes such as cell differentiation in biology ((**?**)). We introduce a set of data generation functions specifically designed to simulate high-dimensional branching structures with various geometries, numbers of points (`n`), and number of branches (`k`). Although these functions can generate multiple branches, they do not produce a formal *multicluster* dataset: the branches form a single connected structure, with multiple visually distinct arms rather than independent clusters. Table @ref(tab:branching-tb-pdf) outlines these functions. The main arguments of the functions described in Table @ref(tab:arg-branching-tb-pdf).

The simplest structures are approximately linear branches in 2-D, generated by the `gen_linearbranches(n, k)` function. These consist of k short line segments in the first two dimensions, with added jitter to

Table 7.2: *cardinalR branching data generation functions*

Function	Explanation
gen_expbranches	Exponential shaped branches.
gen_linearbranches	Linear shaped branches.
gen_curvybranches	Curvy shaped branches.
gen_orglinearbranches	Linear shaped branches originated in one point.
gen_orgcurvybranches	Curvy shaped branches originated in one point.

Table 7.3: *The main arguments for branching shape generators.*

Argument	Type	Explanation
n	integer	Number of points.
k	integer	Number of clusters.

simulate variability. Mathematically, each branch i is defined as

$$X_1 \sim U(a_i, b_i), \quad X_2 = s_i(X_1 - x_{\text{start},i}) + y_{\text{start},i} + \epsilon, \quad \epsilon \sim U(0, \delta),$$

where $(x_{\text{start},i}, y_{\text{start},i})$ is the starting point of branch i, δ controls local jitter, and s_i is the slope, initialized as

$$s_i = \begin{cases} 0.5 & i = 1, \\ -0.5 & i = 2, \\ \text{randomly sampled from } [s_{\min}, s_{\max}] & i = 3, \ldots, k. \end{cases}$$

Branches 1 and 2 are initialized with fixed slopes and intercepts, while later branches are iteratively added at locations chosen to avoid overlap with existing branches, producing a set of connected linear paths.

```
linearbranches <- gen_linearbranches(n = 1000, k = 4)
```

To introduce curvature, the `gen_curvybranches(n, k)` function generates k curvilinear branches in 2-D. Branches 1 and 2 are simple parabolas defined as

$$\text{Branch 1: } X_1 \sim U(0, 1), \quad X_2 = 0.1X_1 + X_1^2 + \epsilon,$$
$$\text{Branch 2: } X_1 \sim U(-1, 0), \quad X_2 = 0.1X_1 - 2X_1^2 + \epsilon, \quad \epsilon \sim U(0, \delta),$$

where δ controls local jitter. Additional branches are attached iteratively to existing structures. Each new branch i starts at a selected point $(x_{\text{start},i}, y_{\text{start},i})$ from the current structure and extends according to

$$X_1 \sim U(x_{\text{start},i}, x_{\text{start},i} + 1), \quad X_2 = 0.1X_1 - s_i(X_1^2 - x_{\text{start},i}) + y_{\text{start},i},$$

where s_i is a scale factor controlling the curvature of branch i. For the first few initial branches, s_i can be fixed (e.g., $s_1 = 1, s_2 = 2$), while for subsequent branches it is sampled from a predefined set, such as $s_i \in \{-2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5\}$, to create variability in curvature.

```
curvybranches <- gen_curvybranches(n = 1000, k = 4)
```

The `gen_expbranches(n, k)` function creates k exponential branches in 2-D, radiating from a central region. Each branch i is defined as

$$X_1 \sim U(-2, 2), \quad X_2 = \exp(\sigma_i s_i X_1) + \epsilon, \quad \epsilon \sim U(0, \delta), \quad s_i \sim U(0.5, 2),$$

where $\sigma_i = (-1)^{i+1}$ alternates the sign of the exponent to produce mirror-symmetric branches. The parameter s_i controls the steepness of branch i, and δ introduces small local jitter.

```
expbranches <- gen_expbranches(n = 1000, k = 4)
```

High-dimensional generalizations are provided by `gen_orglinearbranches(n, p, k)` (Figure @ref(fig:branch-proj)) and `gen_orgcurvybranches(n, p, k)`. Each branch is embedded in a unique or repeated 2-D subspace of the p-D space. When `allow_share = TRUE`, multiple branches may share the same subspace; otherwise, subspaces are sampled without replacement until all possible $\binom{p}{2}$ combinations are exhausted, after which additional branches may repeat subspaces. Linear branches follow

$$X_{i_1} \sim U(a_i, b_i), \quad X_{i_2} = s_i X_{i_1} + \epsilon, \quad \epsilon \sim N(0, \sigma^2),$$

while curvilinear branches include a quadratic term

$$X_{i_1} \sim U(a_i, b_i), \quad X_{i_2} = -s_i X_{i_1}^2 + \epsilon, \quad \epsilon \sim N(0, \sigma^2),$$

where a_i, b_i define the range of the first coordinate for branch i, and ϵ is Gaussian noise added to introduce variability. The scale factor s_i controls slope (linear branches) or curvature (curvilinear branches) and is assigned as follows: for the first $\binom{p}{2}$ branches, $s_i = 1$; for additional branches when $k > \binom{p}{2}$, s_i is randomly drawn from the set $\{1, 1.5, 2, \ldots, 8\}$.

```
orglinearbranches <- gen_orglinearbranches(n = 1000, p = 4, k = 4)
```

```
orgcurvybranches <- gen_orgcurvybranches(n = 1000, p = 4, k = 4)
```

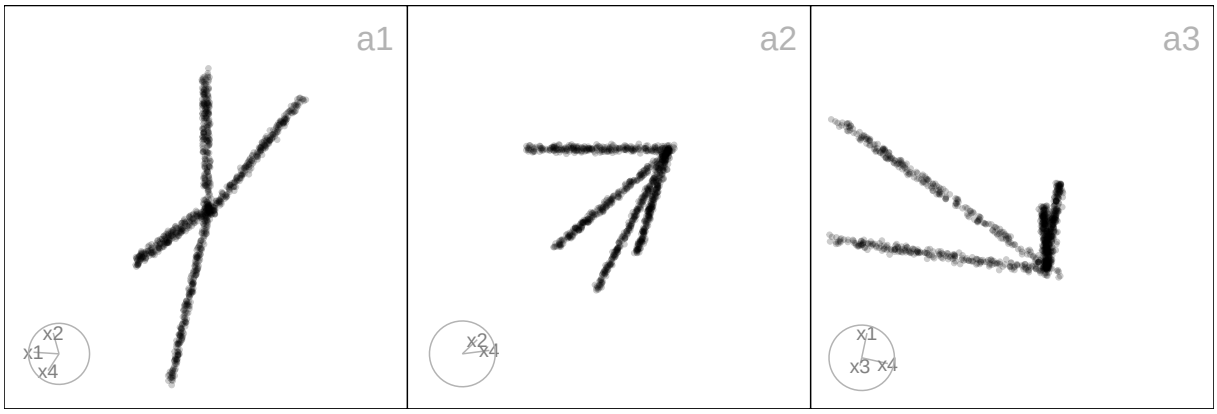

Figure 7.1: *Three 2-D projections from the 4-D* `orglinearbranches` *data. Each shows a different projection, illustrating how the linear branches appear from multiple viewing angles. These views highlight the dataset's underlying branching structure and demonstrate how projections reveal patterns that are otherwise hidden in higher dimensions.*

7.2.2 Cone

To simulate a cone-shaped structure in arbitrary dimensions (Figure @ref(fig:cone-proj)), we define a function `gen_cone(n, p, h, ratio)`, which creates a high-dimensional cone with options for a sharp or blunted apex, allowing for a dense concentration of points near the tip.

This function generates n points in p-D, where the last dimension, X_p, represents the height along the cone's axis, and the first $p-1$ dimensions define a shrinking hyperspherical cross-section toward the tip. Heights are sampled from a truncated exponential distribution, $X_p \sim \text{Exp}(\lambda = 2/h)$, capped at the cone height h, producing a higher density of points near the tip. At each height X_p, the radius of the cross-section decreases linearly from base to tip according to $r = r_{\min} + (r_{\max} - r_{\min})X_p/h$, where $r_{\min} = \text{ratio}$ and $r_{\max} = 1$.

For each point, a direction in the first $p-1$ dimensions is sampled uniformly on a $(p-1)$-dimensional hypersphere using generalized spherical coordinates. The radial coordinates are scaled by the height-dependent radius r, producing the conical taper. In three dimensions ($p = 3$), this results in a classical

Table 7.4: *cardinalR cube data generation functions*

Function	Explanation
gen_gridcube	Cube with specified grid points along each axes.
gen_unifcube	Cube with uniform points.

3-*D* cone, while for $p > 3$, additional dimensions provide a smooth embedding into higher-dimensional space, preserving the conical structure.

```
cone <- gen_cone(n = 1000, p = 4, h = 5, ratio = 0.5)
```

Cone-shaped structures appear in particle dispersions, light beams, and tapering processes, where spread decreases along one axis. They are also used to benchmark clustering and dimensionality reduction methods (**?**).

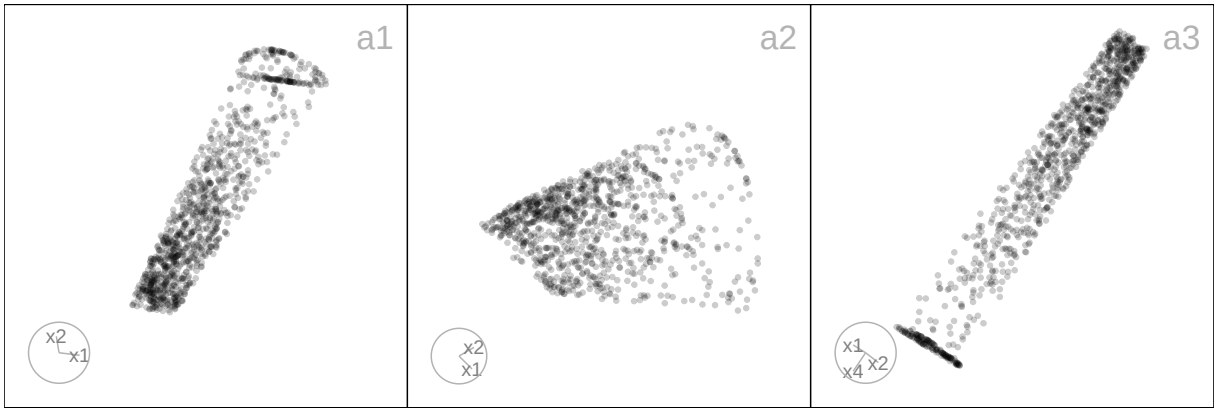

Figure 7.2: *Three 2-D projections from the 3-D* `cone` *data. Points are concentrated near the tip along the height dimension, while the radius of the hyperspherical cross-section decreases linearly toward the apex. These projections show how the conical geometry is preserved.*

7.2.3 Cube

A cube structure represents uniformly or systematically distributed points within a high-dimensional hypercube, providing a useful framework for assessing how well algorithms preserve uniformity, and boundary properties in high dimensions. We provide a set of functions to generate high-dimensional cube structures with flexible configurations, including regular grids, and uniform random points. Table @ref(tab:cube-tb-pdf) outlines these functions and their purposes.

The function `gen_gridcube(n, p)` is a wrapper around `geozoo::cube.solid.grid()`. It generates a regular lattice of points in *p-D*, producing a uniform hypercube grid. Each axis contains equally spaced coordinates, resulting in a well-defined geometric structure.

```
gridcube <- gen_gridcube(n = 1000, p = 4)
```

By contrast, `gen_unifcube(n, p)` wraps `geozoo::cube.solid.random()`, producing uniformly distributed points within a *p-D* cube. To avoid including the cube's vertices, these points are removed after generation. This results in a hypercube filled with random, but evenly distributed, samples rather than structured lattice points.

```
unifcube <- gen_unifcube(n = 1000, p = 4)
```

Such cube-based structures are commonly used as benchmarks in Monte Carlo sampling, computational geometry, and density estimation, where assessing how algorithms behave under uniform or grid-like distributions is critical (**?**; **?**).

7.2.4 Gaussian

The `gen_gaussian(n, p, s)` function generates a multivariate Gaussian cloud in *p-D*, centered at the origin with user-defined covariance structure. Each point is independently drawn using the multivariate normal distribution with $X_i \sim N_p(\mathbf{0}, s)$, where s is a user-defined $p \times p$ positive-definite matrix.

```
gau <- gen_gaussian(n = 1000, p = 4, s = diag(4))
```

Gaussian clouds are common benchmark structures in statistics and machine learning, used in clustering, classification, and anomaly detection, with applications in image segmentation, speech recognition, and forensic analysis (**?**).

7.2.5 Linear

The `gen_longlinear(n, p)` function generates a high-dimensional dataset representing a long linear structure with noise. Each variable is formed as $X_i = \text{scale}_i \cdot (0, 1, \ldots, n{-}1 + \epsilon) + \text{shift}_i$, where scale_$i \sim U(-10, 10)$ determines the orientation of the line in each dimension, shift_$i \sim U(-300, 300)$ offsets the line to separate dimensions, and $\epsilon \sim N(0, (0.03n)^2)$ introduces Gaussian noise.

```
linear <- gen_longlinear(n = 1000, p = 4)
```

This structure appears in *p-D* data when variation is driven by a single factor, such as time-course or sensor measurements, providing a useful test case for trajectory and regression methods (**?**).

Table 7.5: *cardinalR polynomial data generation functions*

Function	Explanation
gen_quadratic	Quadratic pattern.
gen_cubic	Cubic pattern.

7.2.6 Möbius

The `gen_mobius()` function is a wrapper around `geozoo::mobius()`, designed to simplify the generation of a Möbius strip in three dimensions for use in high-dimensional diagnostic studies. The function returns a tibble with n sampled points forming the surface of a Möbius strip.

```
mobius <- gen_mobius(n = 1000)
```

The Möbius strip structure can model twisted or cyclic surfaces in physics and engineering, such as conveyor belts, molecular structures, or optical systems with non-orientable geometries (**?**).

7.2.7 Polynomial

A polynomial structure generates data points that follow non-linear curvilinear relationships, such as quadratic or cubic trends, in 2-D space. To extend these patterns into high-dimensional settings, additional noise dimensions can be added. These patterns are useful for evaluating how well algorithms capture smooth, non-linear trajectories and curvature in the data. We provide functions for generating quadratic and cubic structures, enabling controlled experiments with different degrees of polynomial complexity. Table @ref(tab:polynomial-tb-pdf) summarizes these functions and their purposes.

The first is the quadratic curve of n points in two dimensions. This is generated using `gen_quadratic(n, range)`. The independent variable is defined as $X_1 \sim U(\text{range}[1], \text{range}[2])$, and a raw polynomial basis of degree 2 is applied to form $X_2 = X_1 - X_1^2 + \varepsilon_2$, where $\varepsilon_2 \sim U(0, 0.5)$. This produces a smooth parabolic arc opening downward, with vertical jitter introduced by the noise term.

```
quadratic <- gen_quadratic(n = 1000)
```

The second is the cubic curve of n points in two dimensions. This is generated using `gen_cubic(n, range)`. The independent variable is defined as $X_1 \sim U(\text{range}[1], \text{range}[2])$, and a raw polynomial basis of degree 3 is applied to construct $X_2 = X_1 + X_1^2 - X_1^3 + \varepsilon_2$, where $\varepsilon_2 \sim U(0, 0.5)$. This produces

Table 7.6: *cardinalR pyramid data generation functions*

Function	Explanation
gen_pyrrect	Rectangular-base, with a sharp or blunted apex.
gen_pyrtri	Triangular-base, with a sharp or blunted apex.
gen_pyrstar	Star-shaped base, with a sharp or blunted apex.
gen_pyrfrac	Pyramid with triangular pyramid-shaped holes.

a more complex curvilinear structure than the quadratic case, with both upward and downward turning points.

```
cubic <- gen_cubic(n = 1000)
```

7.2.8 Pyramid

A pyramid structure (Figure @ref(fig:pyr-proj)) represents data arranged around a central apex and base, useful for exploring how algorithms handle pointed or layered geometries in p-D space. The functions provided allow users to generate pyramids with rectangular, triangular, and star-shaped bases, and sharp or blunted apexes. Additionally, it is possible to create a pyramid with a fractal-like internal structure, enabling the study of non-convex and sparse regions. Table @ref(tab:pyramid-tb-pdf) summarizes these functions.

Let $X_1, \ldots, X_p$ denote the coordinates of the generated points. For the rectangular, triangular, and star-shaped based pyramid generator functions, the final dimension, X_p, encodes the height of each point and is drawn from an exponential distribution capped at the maximum height h. That is, $X_p = z \sim \min(\text{Exp}(\lambda = 2/h),\ h)$. This distribution creates a natural skew toward smaller height values, resulting in a denser concentration of points near the pyramid's apex. For the star-shaped base pyramid, the final dimension is drawn from a uniform distribution. That is, $X_p = z \sim U(0, h)$.

The remaining dimensions are based on the specific pyramid shape. For the rectangular based pyramid, `gen_pyrrect(n, p, h, l_vec, rt)` (Figure @ref(fig:pyr-proj) a), let $r_x(z)$ and $r_y(z)$ denote the half-widths of the rectangular cross-section at height z. That is, $r_x(z) = r_t + (l_x - r_t)z/h$, $r_y(z) = r_t + (l_y - r_t)z/h$. The first three coordinates are then defined as $X_1 \sim U(-r_x(z),\ r_x(z))$, $X_2 \sim U(-r_y(z),\ r_y(z))$, and $X_3 \sim U(-r_x(z),\ r_x(z))$.

```
pyrrect <- gen_pyrrect(n = 1000, p = 4)
```

For the triangular based pyramid, `gen_pyrtri(n, p, h, l, rt)` (Figure @ref(fig:pyr-proj) b), let $r(z)$ denote the scaling factor (distance from the origin to triangle vertices) at height z. That is,

$r(z) = r_t + (l - r_t)z/h$. A point in the triangle at height z is generated using barycentric coordinates (u, v) to ensure uniform sampling within the triangular cross-section: $u, v \sim U(0,1), \quad \text{if } u+v>1: u \leftarrow 1-u,\ v \leftarrow 1-v$. The first three coordinates (triangle plane) are then: $X_1 = r(z)(1-u-v)$, $X_2 = r(z)u$, and $X_3 = r(z)v$.

```
pyrtri <- gen_pyrtri(n = 1000, p = 4)
```

For the star based pyramid, `gen_pyrstar(n, p, h, rb)` (Figure @ref(fig:pyr-proj) c), let the radius at height z, $r(z)$, be such that the radius scales linearly from zero (tip) to the base radius r_b. That is, $r(z) = r_b\,(1 - z/h)$. Each point is placed within a regular hexagon in the plane (X_1, X_2), using a randomly chosen hexagon sector angle $\theta \in \{0, \pi/3, 2\pi/3, \pi, 4\pi/3, 5\pi/3\}$ and a uniformly random radial scaling factor: $\theta \sim$ Uniform sample from 6 hexagon angles, $r_{\text{point}} \sim \sqrt{U(0,1)}$. Then, the first two coordinates are: $X_1 = r(z) r_{\text{point}} \cos(\theta)$, and $X_2 = r(z) r_{\text{point}} \sin(\theta)$.

```
pyrstar <- gen_pyrstar(n = 1000, p = 4)
```

For rectangular and triangular pyramids, the remaining dimensions X_4 to X_{p-1}, and for star-based pyramids X_3 to X_{p-1}, are treated as noise.

Finally, for the Sierpinski-like pyramid, `gen_pyrfrac(n, p)` (Figure @ref(fig:pyr-proj) d), let $X_1, X_2, \ldots, X_p$ denote the coordinates of the generated points. The generation process begins with an initial point $T_0 \in [0,1]^p$ drawn from a uniform distribution: $T_0 \sim U(0,1)^p$. Let $C_1, C_2, \ldots, C_{p+1}$ denote the corner vertices of a p-D simplex. At each iteration $i = 1, \ldots, n$, a new point is computed by taking the midpoint between the previous point T_{i-1} and a randomly selected vertex C_k: $T_i = 1/2(T_{i-1} + C_k), \quad C_k \in \{C_1, \ldots, C_{p+1}\}$. This recursive midpoint rule generates self-similar patterns with systematic voids (holes) between clusters of points. The points remain bounded inside the convex hull of the simplex. The final output is a $n \times p$ matrix where each row represents a point: $X = \{T_1, T_2, \ldots, T_n\}, \quad X \in \mathbb{R}^{n \times p}$.

```
pyrholes <- gen_pyrfrac(n = 1000, p = 4)
```

Pyramid structures mimic tapering or layered geometries seen in architecture, crystals, and fractal-like natural patterns (**?**).

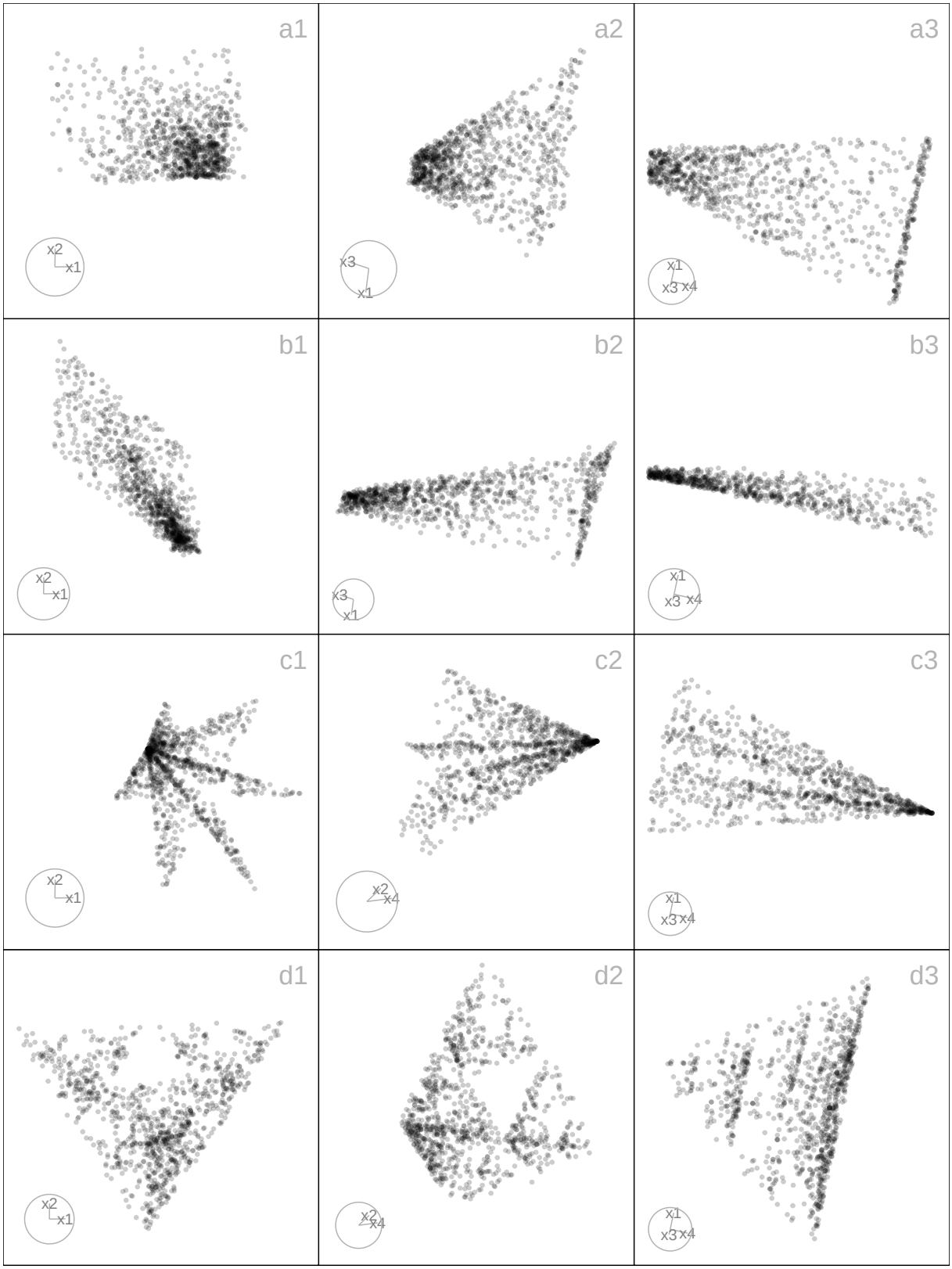

Figure 7.3: *Three 2-D projections from 4-D, for the* `pyrrect` *(a1-a3),* `pyrtri` *(b1-b3),* `pyrstar` *(c1-c3), and* `pyrholes` *(d1-d3) data. The* `pyrrrect` *structure forms a dense rectangular base tapering to a narrow tip, while* `pytri` *shows a more triangular spread with sharper edges.* `Pyrstar` *extends into multiple pointed branches radiating from a common core, and* `pyrholes` *reveals hollow or open regions within an otherwise compact shape. These projections illustrate a range of pyramid-like geometries that vary in density and structure.*

Table 7.7: *cardinalR sphere data generation functions*

Function	Explanation
gen_circle	Circle.
gen_curvycycle	Curvy cell cycle.
gen_unifsphere	Uniform sphere.
gen_hollowsphere	Hollow sphere.
gen_gridedsphere	Grided sphere.
gen_clusteredspheres	Multiple small spheres within a big sphere.
gen_hemisphere	Hemisphere.

7.2.9 S-curve

The S-curve is a smooth, non-linear manifold in 3-D space. Using `gen_scurve(n)`, it is defined by

$$X_1 = \sin(\theta), \quad X_2 \sim U(0,2), \quad X_3 = \text{sign}(\theta)(\cos(\theta) - 1), \quad \theta \sim U(-3\pi/2, 3\pi/2).$$

This follows the `s_curve()` function from snedata (**?**), itself adapted from *scikit-learn*, but differs by returning a tibble with standardized names (`x1`, `x2`, `x3`), excluding the color variable, and omitting built-in noise (which can be added separately). S-curve is commonly used in manifold learning and dimension reduction as benchmarks for unfolding curved structure.

```
scurve <- gen_scurve(n = 1000)
```

7.2.10 Sphere

Sphere-shaped structures are useful for evaluating how dimension reduction and clustering algorithms handle curved, symmetric manifolds in high-dimensional spaces. The functions generate a variety of spherical forms, including simple circles, uniform and hollow spheres, grid-based spheres, and complex arrangements like clustered spheres within a larger sphere. The first few coordinates define the main geometric form (circle, cycle, sphere, or hemisphere), while higher-dimensional embeddings are achieved by adding noise dimensions. Such spherical or hemispherical structures frequently appear in physical and biological systems, for example in models of celestial bodies, molecular shells, or cell membranes (**?**; **?**).Table @ref(tab:sphere-tb-pdf) summarizes these functions.

The simplest case, `gen_circle(n, p)` creates a unit circle in two dimensions, with the remaining dimensions forming sinusoidal extensions of the angular parameter at progressively smaller scales (Figure @ref(fig:sphere-proj) a). Let a latent angle variable θ is uniformly sampled from the interval $[0, 2\pi]$. Coordinates in the first two dimensions represent a perfect circle on the plane:

$$X_1 = \cos(\theta), \quad X_2 = \sin(\theta).$$

For dimensions X_3 through X_p, sinusoidal transformations of the angle θ are introduced. The first component is a scaling factor that decreases with the dimension index, defined as $\text{scale}_j = \sqrt{(0.5)^{j-2}}$ for $j = 3, \ldots, p$. The second component is a phase shift that is proportional to the dimension index, specifically designed to decorrelate the curves, given by the formula $\phi_j = (j-2)\pi/2p$. Each additional dimension is computed as: $X_j = \text{scale}_j \sin(\theta + \phi_j), \quad j = 3, \ldots, p$.

```
circle <- gen_circle(n = 1000, p = 4)
```

For the one-dimensional nonlinear cycle embedded in p-D space, `gen_curvycycle(n, p)` (Figure @ref(fig:sphere-proj) b), let a latent angle variable θ is uniformly sampled from the interval $[0, 2\pi]$. The first three dimensions define a non-circular closed curve, referred to as a "curvy cycle". In this configuration, $X_1 = \cos(\theta)$ represents horizontal oscillation, while $X_2 = \sqrt{3}/3 + \sin(\theta)$ introduces a vertical offset to avoid centering the curve at the origin. Additionally, $X_3 = 1/3\cos(3\theta)$ introduces a third harmonic perturbation that intricately folds the curve three times along its path, creating a unique and complex shape that oscillates in both dimensions while incorporating the effects of the harmonic perturbation.

Together, these define a periodic, non-trivial, closed curve in 3-D with internal folds that produce a more complex geometry than a standard circle or ellipse. For dimensions X_4 through X_p, additional structured variability is introduced through decreasing amplitude scaling and phase-shifted sine waves. The scaling factor is defined as $\text{scale}_j = \sqrt{(0.5)^{j-3}}$ for j ranging from 4 to p, which means that the amplitude decreases as the dimension increases. Each dimension X_j is then calculated using the formula $X_j = \text{scale}_j \sin(\theta + \phi_j)$, where the phase shift ϕ_j is given by $\phi_j = (j-2)\pi/2p$.

```
curvycycle <- gen_curvycycle(n = 1000, p = 4)
```

Building on simple circular structures, the `gen_unifsphere(n, r)` function function extends the idea to three dimensions by generating n observations approximately uniformly distributed on the surface of a sphere of radius r.. Each observation is computed from spherical coordinates, with $u \sim U(-1, 1)$ representing $\cos(\phi)$ and $\theta \sim U(0, 2\pi)$ the azimuthal angle. Cartesian coordinates are then defined as

$$X_1 = r\sqrt{1-u^2}\cos(\theta), \quad X_2 = r\sqrt{1-u^2}\sin(\theta), \text{ and } X_3 = ru,$$

ensuring uniform distribution on the surface (not within) of the sphere.

```
unifsphere <- gen_unifsphere(n = 1000, r = 1)
```

In contrast, the `gen_hollowsphere(n, p)` function, a wrapper around `geozoo::sphere.hollow()`, generates n points uniformly distributed only on the surface of the $(p-1)$-dimensional sphere embedded in $\mathbb{R}^p$. This results in a hollow shell-like structure with no interior points. For example, when $p = 3$, `gen_unifsphere()` produces a solid ball in 3-D space, whereas `gen_hollowsphere()` produces only the spherical boundary. These paired structures allow controlled experiments to investigate how algorithms behave when data is concentrated throughout the full volume versus constrained to the boundary.

```
hollowsphere <- gen_hollowsphere(n = 1000, p = 4)
```

In addition, the `gen_gridedsphere(n)` function constructs a p-D dataset consisting of approximately n points that are evenly distributed on the surface of the unit $(p-1)$-sphere embedded in $\mathbb{R}^p$ (Figure @ref(fig:sphere-proj) d). The method relies on forming a regular grid in spherical coordinates, parameterized by $(p-1)$ angular variables: for dimensions $j = 1, \ldots, p-2$ the polar angles are drawn from $[0, \pi]$, while the final angle ($j = p-1$) represents the azimuth and is drawn from $[0, 2\pi]$. The number of grid steps along each angular dimension is chosen by decomposing n into $(p-1)$ approximately equal integer factors using the helper function `gen_nproduct(n, p - 1)`.

Each grid point is subsequently mapped into Cartesian space via the standard hyperspherical-to-Cartesian transformation,

$$
\begin{aligned}
X_1 &= \cos(\theta_1), \\
X_2 &= \sin(\theta_1)\cos(\theta_2), \\
X_3 &= \sin(\theta_1)\sin(\theta_2)\cos(\theta_3), \\
&\vdots \\
X_{p-1} &= \sin(\theta_1)\sin(\theta_2)\cdots\sin(\theta_{p-2})\cos(\theta_{p-1}), \\
X_p &= \sin(\theta_1)\sin(\theta_2)\cdots\sin(\theta_{p-2})\sin(\theta_{p-1}).
\end{aligned}
$$

The result is a deterministic grid of points lying exactly on the surface of the unit $(p-1)$-sphere, without any additional noise dimensions.

For more heterogeneous structures, the `gen_clusteredspheres(n, k, r, loc)` function generates one large sphere of radius r_1 and k smaller spheres of radius r_2, each centered at a different random location (Figure @ref(fig:sphere-proj) e). A large uniform sphere centered at the origin is

created by sampling n_1 points uniformly on the surface of a p-D sphere with a radius of r_1. The sampling is executed using the function `gen_unifsphere(n_1, r_1)`, which generates the desired points in the specified dimensional space. In generation of k smaller uniform spheres, each sphere contains n_2 points that are sampled uniformly on a sphere with a radius of r_2. These spheres are positioned at distinct random locations in p-space, with the center of each sphere being drawn from a normal distribution $N(0, \texttt{loc}^2 I_p)$. Points on spheres are generated using the standard hyperspherical method, which involves sampling $u \sim U(-1, 1)$ to determine the cosine of the polar angle, and sampling $\theta \sim U(0, 2\pi)$ to determine the azimuthal angle (for 3-D). Each observation is classified by cluster, with labels such as "big" for the large central sphere and "small_1" to "small_k" for the smaller spheres.

```
clusteredspheres <- gen_clusteredspheres(n = c(1000, 100), k = 3, r = c(15, 3),
                                         loc = 10 / sqrt(3)) |>
  dplyr::select(-cluster)
```

Finally, the `gen_hemisphere(n, p)` function restricts sampling to a hemisphere of a 4-D sphere (Figure @ref(fig:sphere-proj) f). Using spherical coordinates, the azimuthal angle $\theta_1 \sim U(0, \pi)$ in the (x_1, x_2) plane, while the elevation angle $\theta_2 \sim U(0, \pi)$ in the (x_2, x_3) plane. Additionally, $\theta_3 \sim U(0, \pi/2)$ in the (x_3, x_4) plane, ensuring that the points remain restricted to a hemisphere. The coordinates are transformed into 4-D Cartesian space:

$$X_1 = \sin(\theta_1)\cos(\theta_2), \quad X_2 = \sin(\theta_1)\sin(\theta_2), \quad X_3 = \cos(\theta_1)\cos(\theta_3), \quad X_4 = \cos(\theta_1)\sin(\theta_3).$$

This produces points on one side of a 4-D unit sphere, effectively generating a 4-D hemisphere.

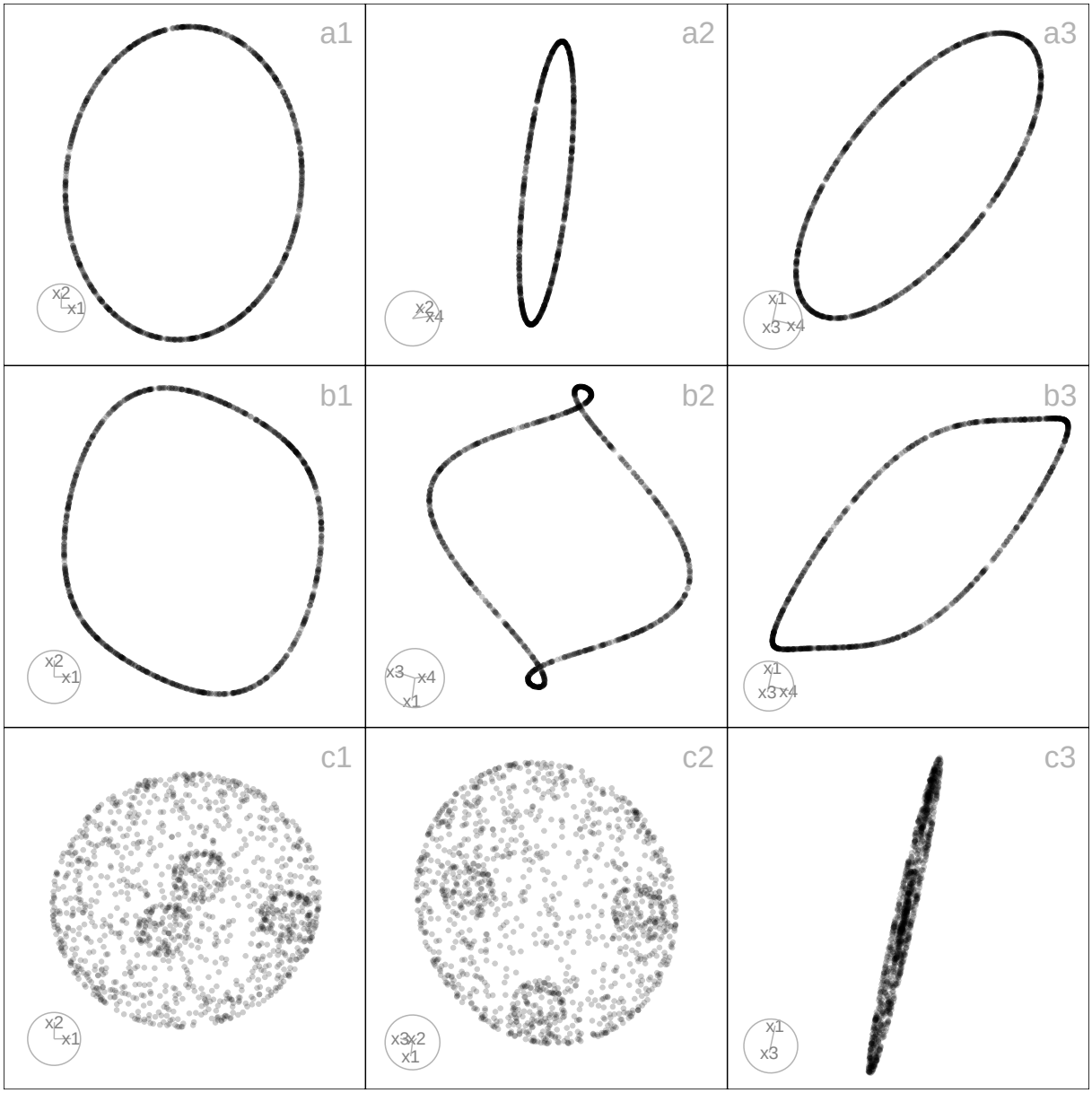

Figure 7.4: *Three 2-D projections from 4-D,* `circle` *(a1-a3),* `curvycycle` *(b1-b3), and, 3-D* `clusteredspheres` *(c1-c3). The* `circle` *structure forms a smooth, closed loop, while* `curvycycle` *shows a wavy, continuous pattern forming a twisted ring. The* `clusteredspheres` *dataset displays multiple compact spherical groups that are clearly separated in higher dimensions but overlap slightly in some 2-D projections, highlighting how projection can distort spatial relationships. These projections show how simple cyclic, wavy curvilinear, and clustered structures appear in 2-D, emphasizing the effects of projection on density, continuity, and separation*

7.2.11 Swiss Roll

The Swiss roll is a standard nonlinear manifold, representing a 2-D plane curled into 3-D. The `gen_swissroll(n, w)` generates points as $X_1 = t\cos(t), \quad X_2 = t\sin(t), \quad X_3 \sim U(w_1, w_2), \quad t \sim U(0, 3\pi)$.

Table 7.8: *cardinalR trefoil data generation functions*

Function	Explanation
gen_trefoil4d	Trefoil in 4-D.
gen_trefoil3d	Trefoil in 3-D.

```
swissroll <- gen_swissroll(n = 1000, w = c(-1, 1))
```

Compared with `sndata::swiss_roll()` (**?**), this implementation (i) samples t over $[0, 3\pi]$ instead of $[1.5\pi, 4.5\pi]$, (ii) allows a flexible vertical range $w = (w_1, w_2)$ rather than fixing $z \in [0, z_{\max}]$, and (iii) returns a tibble with `x1, x2, x3` instead of adding a color variable.

The Swiss roll is a classic benchmark for manifold learning, illustrating how a curved surface can be "unrolled" into lower dimensions. Similar spiral-like forms appear in galaxies, protein folding, and coiled materials (**?**).

7.2.12 Trefoil knots

The Trefoil is a closed, nontrivial one-dimensional manifold embedded in 3-*D* or 4-*D* space (Figure @ref(fig:trefoil-proj)). The trefoil features topological complexity in the form of self-overlaps, making it a valuable test case for evaluating the ability of non-linear dimension reduction methods to preserve global structure, loops, and embeddings in high-dimensional data. Table @ref(tab:trefoil-tb-pdf) summarizes these functions.

For the 4-*D* trefoil knot, the function `gen_trefoil4d(n, steps)` generates the structure on the 3-sphere ($S^3 \subset \mathbb{R}^4$) using two angular parameters, θ and ϕ. A band of thickness around the knot path is controlled by the `steps` argument, while the number of θ and ϕ values is determined by the `steps` and `n` arguments, respectively (Figure @ref(fig:trefoil-proj) a). The coordinates are defined as

$$X_1 = \cos(\theta)\cos(\phi), \quad X_2 = \cos(\theta)\sin(\phi), \quad X_3 = \sin(\theta)\cos(1.5\phi), \text{ and } X_4 = \sin(\theta)\sin(1.5\phi),$$

where θ and ϕ trace the knot's path.

```
trefoil4d <- gen_trefoil4d(n = 500, steps = 5)
```

For the 3-*D* stereographic projection, `gen_trefoil3d(n, steps)` maps each point $(X_1, X_2, X_3, X_4) \in \mathbb{R}^4$ to $(X_1', X_2', X_3') \in \mathbb{R}^3$ using $X_1' = X_1/(1-X_4)$, $X_2' = X_2/(1-X_4)$, and $X_3' = X_3/(1-X_4)$, excluding points where $X_4 = 1$ to avoid division by zero (Figure @ref(fig:trefoil-proj) b).

```
trefoil3d <- gen_trefoil3d(n = 500, steps = 5)
```

The trefoil knot appears in molecular biology (DNA/protein knotting), fluid dynamics (knotted vortices), and physics (topological phases), making it a useful benchmark for testing whether dimension reduction preserves global loops and topology (**?**; **?**).

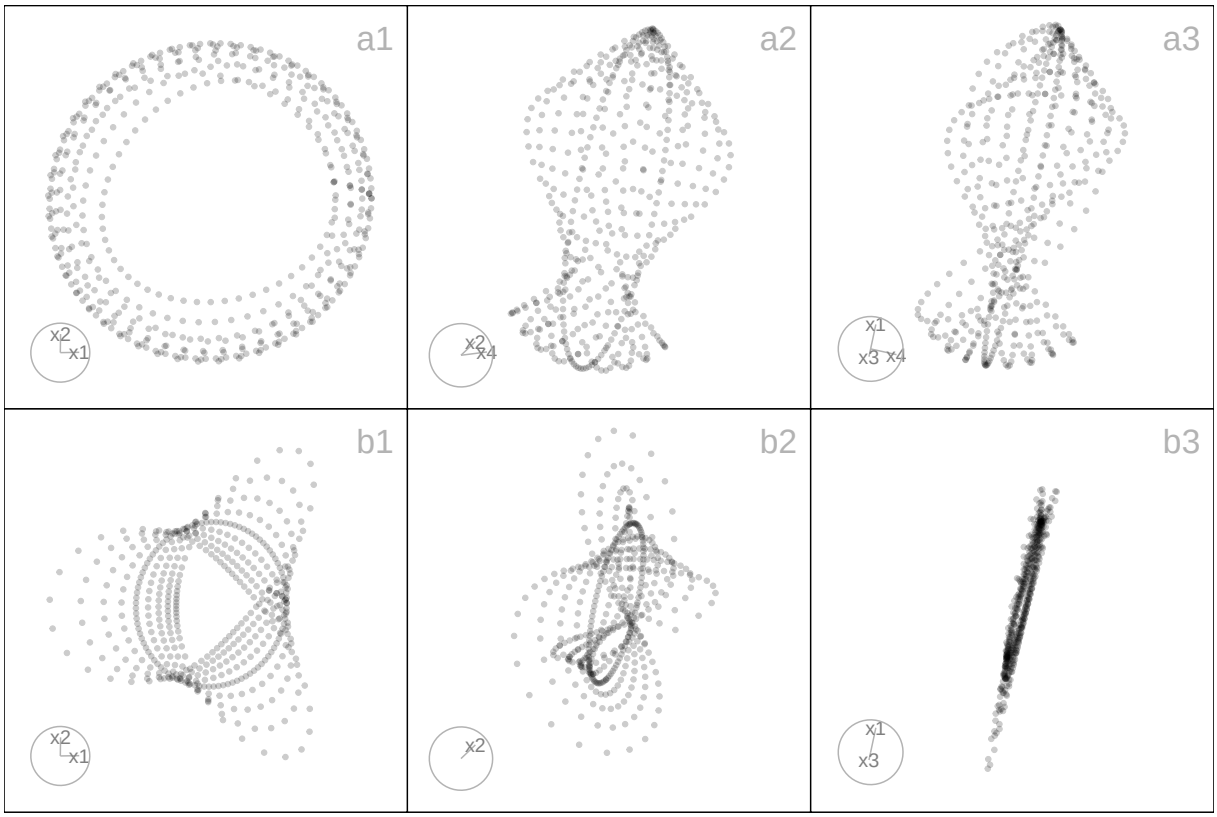

Figure 7.5: *Three 2-D projections from 4-D, `trefoil4d` (a1-a3) and 3-D `trefoil3d` (b1-b3) data. The `trefoil4d` structure represents a higher-dimensional extension of the classic trefoil knot, revealing complex twisting and looping patterns that remain continuous across projections. In contrast, the `trefoil3d` dataset maintains a simpler, more compact knot-like form, showing how dimensional extension adds curvature and separation in the embedded space. These projections illustrate a range of looping structures in high-dimensions.*

7.2.13 Trigonometric

Trigonometric-based structures provide flexible ways to simulate complex curved patterns and spirals that often arise in real-world high-dimensional data, such as in biological trajectories, or physical systems (Figure @ref(fig:triginometric-proj)). The main geometry is defined by the first few coordinates: crescents ($p = 2$), cylinders, spirals, and helices ($p = 4$). These structures are particularly valuable for testing how well dimension reduction and clustering algorithms preserve intricate geometric and topological features (**?**; **?**). Table @ref(tab:trigonometric-tb-pdf) summarizes these functions.

Table 7.9: *cardinalR trigonometric data generation functions*

Function	Explanation
gen_crescent	Crescent pattern.
gen_curvycylinder	Curvy cylinder.
gen_sphericalspiral	Spherical spiral.
gen_helicalspiral	Helical spiral.
gen_conicspiral	Conic spiral.
gen_nonlinear	Nonlinear hyperbola.

First, the `gen_crescent(n, p)` function generates a p-dimensional dataset of n observations based on a 2-D crescent-shaped manifold with optional structured high-dimensional noise (Figure @ref(fig:triginometric-proj) a). Let $\theta \in [\pi/6, 2\pi]$ be a sequence of n evenly spaced angles. The corresponding 2-D coordinates are defined by:

$$X_1 = \cos(\theta), \quad X_2 = \sin(\theta).$$

```
crescent <- gen_crescent(n = 1000)
```

Second, the `gen_curvycylinder(n, p, h)` function generates a p-dimensional dataset of n observations structured as a 3-D cylindrical manifold with an added nonlinear curvy dimension, and optional noise dimensions when $p > 4$ (Figure @ref(fig:triginometric-proj) b). The core structure consists of a circular base and height values, extended by a nonlinear fourth dimension. Let $\theta \sim U(0, 3\pi)$ represent a random angle on a circular base and $z \sim U(0, h)$ represent the height along the cylinder. The coordinates are defined as: $X_1 = \cos(\theta)$ (Circular base, x-axis), $X_2 = \sin(\theta)$ (Circular base, y-axis), $X_3 = z$ (Linear height), and $X_4 = \sin(z)$ (Nonlinear curvy variation along height).

```
curvycylinder <- gen_curvycylinder(n = 1000, h = 10)
```

For a spiraling path on a spherical surface in the first four dimensions, `gen_sphericalspiral(n, p, spins)` (Figure @ref(fig:triginometric-proj) c), let $\theta \in [0, 2\pi \times \text{spins}]$ be the azimuthal angle (longitude), controls the number of spiral turns and the $\phi \in [0, \pi]$be the polar angle (latitude), controls the vertical sweep from the north to the south pole. Cartesian coordinates from spherical conversion: $X_1 = \sin(\phi)\cos(\theta)$, $X_2 = \sin(\phi)\sin(\theta)$, $X_3 = \cos(\phi) + \varepsilon$, where $\varepsilon \sim U(-0.5, 0.5)$ introduces vertical jitter, and $X_4 = \theta / \max(\theta)$: a normalized progression along the spiral path. This generates a spherical spiral curve embedded in 4-D space, combining both circular and vertical movement, with gentle curvature and non-linear progression.

```
sphericalspiral <- gen_sphericalspiral(n = 1000, spins = 1)
```

For a helical spiral in four dimensions, `gen_helicalspiral(n, p)` (Figure @ref(fig:triginometric-proj) d), let $\theta \in [0, 5\pi/4]$ be a sequence of angles controlling rotation around a circle. Cartesian coordinates; $X_1 = \cos(\theta)$: circular trajectory along the x-axis, $X_2 = \sin(\theta)$: circular trajectory along the y-axis, $X_3 = 0.05\theta + \varepsilon_3$, with $\varepsilon_3 \sim U(-0.5, 0.5)$: linear progression (height) with vertical jitter, simulating a helix, and $X_4 = 0.1\sin(\theta)$: oscillates with θ, representing a periodic "wobble" along the fourth dimension.

```
helicalspiral <- gen_helicalspiral(n = 1000)
```

Similarly, the `gen_conicspiral(n, p, spins)` function generates a dataset of n points forming a conical spiral in the first four dimensions of p-D (Figure @ref(fig:triginometric-proj) e). The geometry combines radial expansion, vertical elevation, and spiral deformation, simulating a structure that fans out like a 3-D conic helix. The shape is defined by parameter $\theta \in [0, 2\pi\text{spins}]$, controlling the angular progression of the spiral. The Archimedean spiral in the horizontal plane is represented by; $X_1 = \theta\cos(\theta)$ for radial expansion in x, and $X_2 = \theta\sin(\theta)$ for radial expansion in y. The growth pattern resembles a cone, with the height increasing according to $X_3 = 2\theta/\max(\theta) + \varepsilon_3$, with $\varepsilon_3 \sim U(-0.1, 0.6)$. Spiral modulation in the fourth dimension is represented by $X_4 = \theta\sin(2\theta) + \varepsilon_4$, with $\varepsilon_4 \sim U(-0.1, 0.6)$ which simulates a twisting helical component in a non-radial dimension.

```
conicspiral <- gen_conicspiral(n = 1000, spins = 1)
```

Finally, the `gen_nonlinear(n, p, hc, non_fac)` function simulates a non-linear 2-D surface embedded in higher dimensions, constructed using inverse and trigonometric transformations applied to independent variables (Figure @ref(fig:triginometric-proj) f). The $X_1 \sim U(0.1, 2)$: base variable (avoids zero to prevent division errors), $X_3 \sim U(0.1, 0.8)$: independent auxiliary variable, $X_2 = hc/X_1 + \text{nonfac}\sin(X_1)$: non-linear combination of hyperbolic and sinusoidal transformations, creating sharp curvature and oscillation, and $X_4 = \cos(\pi X_1) + \varepsilon$, with $\varepsilon \sim U(-0.1, 0.1)$: additional nonlinear variation based on cosine, simulating more subtle periodic structure. These transformations together result in a non-linear surface warped in multiple ways: sharp vertical shifts due to inverse terms, smooth waves from sine and cosine, and additional jitter.

```
nonlinear <- gen_nonlinear(n = 1000, hc = 1, non_fac = 0.5)
```

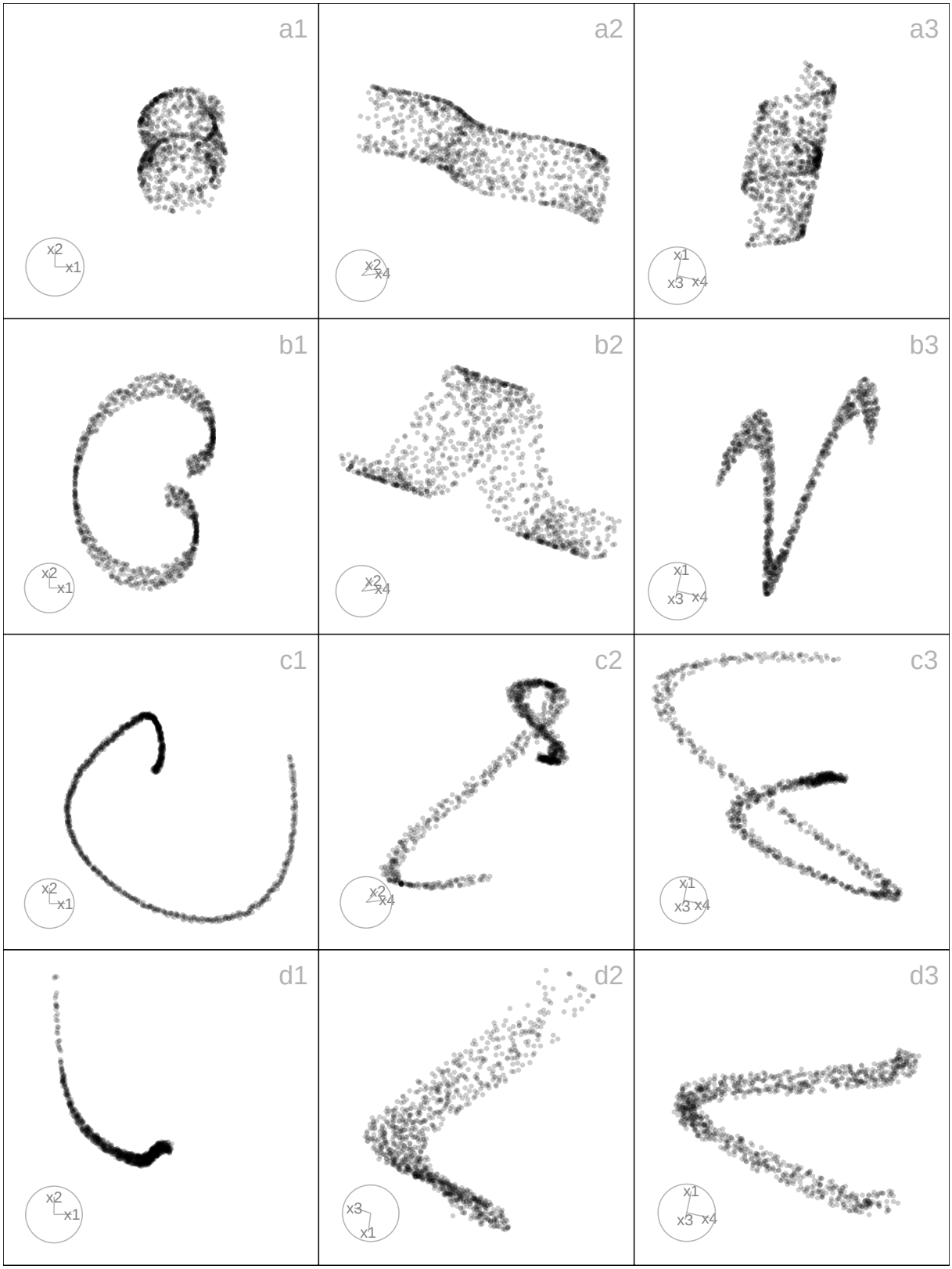

Figure 7.6: *Three 2-D projections from 4-D, for the* `curvycylinder` *(a1-a3),* `sphericalspiral` *(b1-b3),* `conicspiral` *(c1-c3), and* `nonlinear` *(d1-d3) data. The* `curvycylinder` *shows a cylindrical manifold with a nonlinear twist along its height, producing smooth, continuous curvature. The* `sphericalspiral` *forms a spiral path on a spherical surface, combining circular and vertical motion in a helical form. The* `conicspiral` *spreads radially while ascending, forming a conical helix with twisting variations in a non-radial dimension. The* `nonlinear` *dataset exhibits a warped 2-D surface with sharp oscillations and smooth waves, reflecting complex nonlinear interactions. Each shows variations in curvature, density, and continuity.*

7.3 Generate a spherical or hyperspherical hole within a structure

The package provides functionality for generating datasets with spherical hole (in 2-D/3-D) or, more generally, hyperspherical hole (in higher dimensions). These structures are valuable for evaluating how dimension reduction methods and clustering algorithms handle incomplete manifolds or missing regions of the data space. A hyperspherical hole introduces topological complexity: the structure remains continuous but contains excluded regions (voids) that algorithms must correctly represent in lower-dimensional embeddings.

The core function `gen_hole(df, anchor, r)` removes points from a dataset that fall within a user-specified hypersphere. Formally, given data points ($x \in \mathbb{R}^p$), a center ($a \in \mathbb{R}^p$), and radius ($r > 0$), only points satisfying $||x - a||_2 > r$ are retained. The anchor point (a) can either be user-specified or default to the dataset mean, and radius (r) is controlled by the user, with safeguards to avoid trivial or degenerate cases. Because it operates generically on any dataset, spherical or hyperspherical holes can be embedded in a wide range of geometric structures.

Two specialized wrappers illustrate this idea. The function `gen_scurvehole(n, r_hole)` generates an S-curve with a spherical hole by applying `gen_hole()` to the output of `gen_scurve()`. This structure has been used in prior diagnostic studies of NLDR methods, since it tests the ability of algorithms to capture non-linear manifolds that are not simply connected. The second wrapper, `gen_unifcubehole(n, p, r_hole)`, generates uniformly sampled cube data with a hyperspherical hole. By embedding a hyperspherical void inside a convex high-dimensional structure, this creates non-convex regions that challenge algorithms in terms of separability and neighborhood preservation.

7.4 Generate noise dimensions

High-dimensional data structures often benefit from the addition of auxiliary noise dimensions, which can be used to assess the robustness of dimensionality reduction and clustering algorithms. The functions in this section provide flexible ways to generate random noise dimensions, ranging from purely random Gaussian variables to more structured, wavy patterns that mimic non-linear distortions in high-dimensional space. These functions can be applied independently or combined with other geometric structures to create complex simulated datasets. Table @ref(tab:noise-tb-pdf) details these functions.

The `gen_noisedims(n, p, m, s)` function generates p independent Gaussian noise dimensions,

Table 7.10: *cardinalR noise dimensions generation functions*

Function	Explanation
gen_noisedims	Gaussian noise dimensions with optional mean and standard deviation.
gen_wavydims1	Wavy noise dimensions based on a user-specified theta sequence with added jitter.
gen_wavydims2	Wavy noise dimensions using polynomial transformations of an existing dimension vector.
gen_wavydims3	Wavy noise dimensions using a combination of polynomial and sine transformations based on the first three dimensions of a dataset.

$$X_j \sim N(m_j, s_j^2), \quad j = 1, \ldots, p,$$

with odd-numbered dimensions multiplied by -1 to introduce sign alternation, enhancing variability and decorrelation.

For scenarios where noise should follow a smooth wavy pattern, `gen_wavydims1(n, p, theta)` generates dimensions as

$$X_j = \alpha_j \theta + \varepsilon_j, \quad \varepsilon_j \sim N(0, \sigma^2), \quad j = 1, \ldots, p,$$

where each dimension is scaled by a different factor α_j, producing structured noise that oscillates along the latent parameter θ, mimicking trends or trajectories observed in real-world data.

The `gen_wavydims2(n, p, x_1)` function extends this approach by applying a non-linear transformation to an existing dimension vector x_1:

$$X_j = \beta_j (-1)^{\lfloor j/2 \rfloor} x_1^{k_j} + \varepsilon_j, \quad j = 1, \ldots, p,$$

where k_j is a randomly chosen polynomial power, β_j is a scaling factor, and ε_j is small uniform noise.

Finally, `gen_wavydims3(n, p, data)` generates noise for datasets with multiple correlated dimensions. The first three dimensions are small perturbations of the original coordinates (X_1, X_2, X_3), while higher dimensions are constructed via non-linear combinations, including polynomial and trigonometric transformations, e.g.,

$$X_j = f_j(X_1, X_2, X_3) + \varepsilon_j, \quad j > 3,$$

Table 7.11: *cardinalR multiple clusters generation functions*

Function	Explanation
make_mobiusgau	Möbius-like cluster combined with a Gaussian.
make_multigau	Multiple Gaussian clusters in high-dimensional space.
make_curvygau	Curvilinear cluster with a Gaussian cluster.
make_klink_circles	K-link circular clusters (non-linear circular patterns).
make_chain_circles	Chain-like circular clusters connected sequentially.
make_klink_curvycycle	K-link curvy cycle clusters (curvilinear loop structures).
make_chain_curvycycle	Chain-like curvy cycle clusters connected sequentially.
make_gaucircles	Circular clusters with a Gaussian cluster in the middle.
make_gaucurvycycle	Curvy circular clusters with a Gaussian in the middle.
make_onegrid	Single grid in two dimensions.
make_twogrid_overlap	Two overlapping grids.
make_twogrid_shift	Two grids shifted relative to each other.
make_shape_para	Parallel shaped clusters.
make_three_clust_	Three clusters with different shapes. (eg:- 01, 02, ..., 20)

producing high-dimensional noise that preserves some geometric correlation with the base structure while introducing additional complexity.

7.5 Multiple cluster examples

By using the shape generators mentioned above, we can create various examples of multiple clusters. The package includes some of these examples, which are described in Table @ref(tab:odd-shape-tb-pdf).

7.6 Additional functions

The package includes various supplementary tools in addition to the shape generating functions mentioned earlier. These tools allow users to create background noise, randomize the rows of the data, relocate clusters, generate a vector whose product and sum are approximately equal to a target value, rotate structures, and normalize the data. Table @ref(tab:add-tb-pdf) details these functions.

Table 7.12: *cardinalR additional functions*

Function	Explanation
gen_bkgnoise	Adds background noise.
randomize_rows	Randomizes the rows.
relocate_clusters	Relocates the clusters.
gen_nproduct	Generates a vector of positive integers whose product is approximately equal to a target value.
gen_nsum	Generates a vector of positive integers whose summation is approximately equal to a target value.
gen_rotation	Generates rotations.
normalize_data	Normalizes data.

Chapter 8

Application

This section demonstrates how the package can be used to generate complex high-dimensional datasets, apply dimension reduction (DR) techniques, and evaluate clustering performance. The example shows how diverse geometric structures can be simulated and analyzed to assess algorithmic behavior.

8.1 Generating high-dimensional clustered data

To illustrate, we generate a dataset with five clusters in 4-D, each representing distinct geometric characteristics: a *helical spiral* (elongated and twisted), a *hemisphere* (curved surface), a *uniform cube* (isotropic distribution), a *cone* (density gradient), and a *Gaussian* cluster (compact and spherical) (Figure @ref(fig:highd-proj)). Each cluster has a unique number of points and scaling factor, representing variation in cluster size and spread across the 4-D space.

```
positions <- geozoo::simplex(p=4)$points
positions <- positions * 0.3

## To generate data
five_clusts <- gen_multicluster(n = c(2250, 1500, 750, 1250, 1750), k = 5,
                        loc = positions,
                        scale = c(0.25, 0.35, 0.3, 1, 0.3),
                        shape = c("helicalspiral", "hemisphere", "unifcube",
                                  "cone", "gaussian"),
```

```
                    rotation = NULL,
                    is_bkg = FALSE)
```

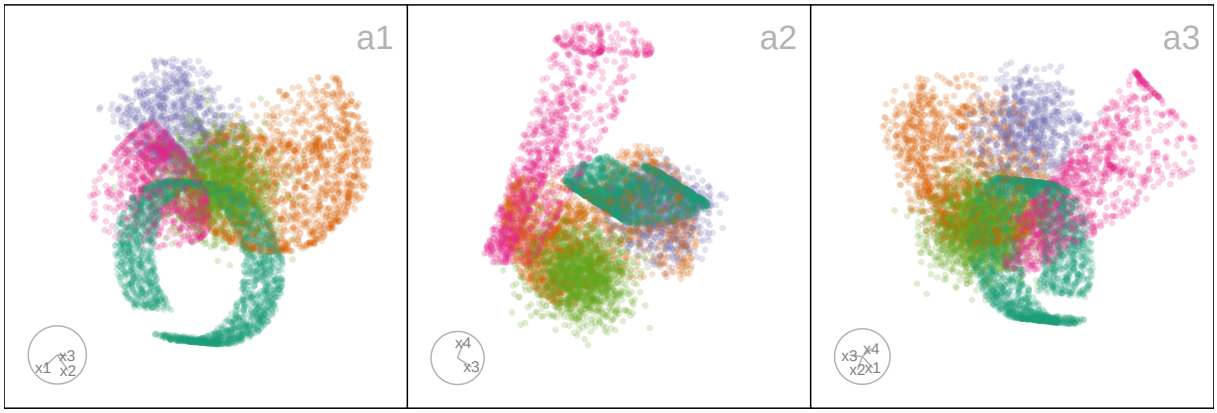

Figure 8.1: *Three 2-D projections from 4-D, for the five clusters data. The helical spiral cluster is represented in dark green, the hemisphere cluster in orange, the uniform cube-shaped cluster in purple, the blunted cone cluster in pink, and the Gaussian-shaped cluster in light green.*

8.2 Evaluating dimension reduction (DR) methods

We applied six popular DR techniques to the generated dataset: Principal Component Analysis (PCA) (Jolliffe 2011), t-distributed stochastic neighbor embedding (tSNE) (Maaten and Hinton 2008), uniform manifold approximation and projection (UMAP) (McInnes et al. 2018), potential of heat-diffusion for affinity-based trajectory embedding (PHATE) algorithm (Moon et al. 2019), large-scale dimensionality reduction Using triplets (TriMAP) (Amid and Warmuth 2019), and pairwise controlled manifold approximation (PaCMAP) (Wang et al. 2021).

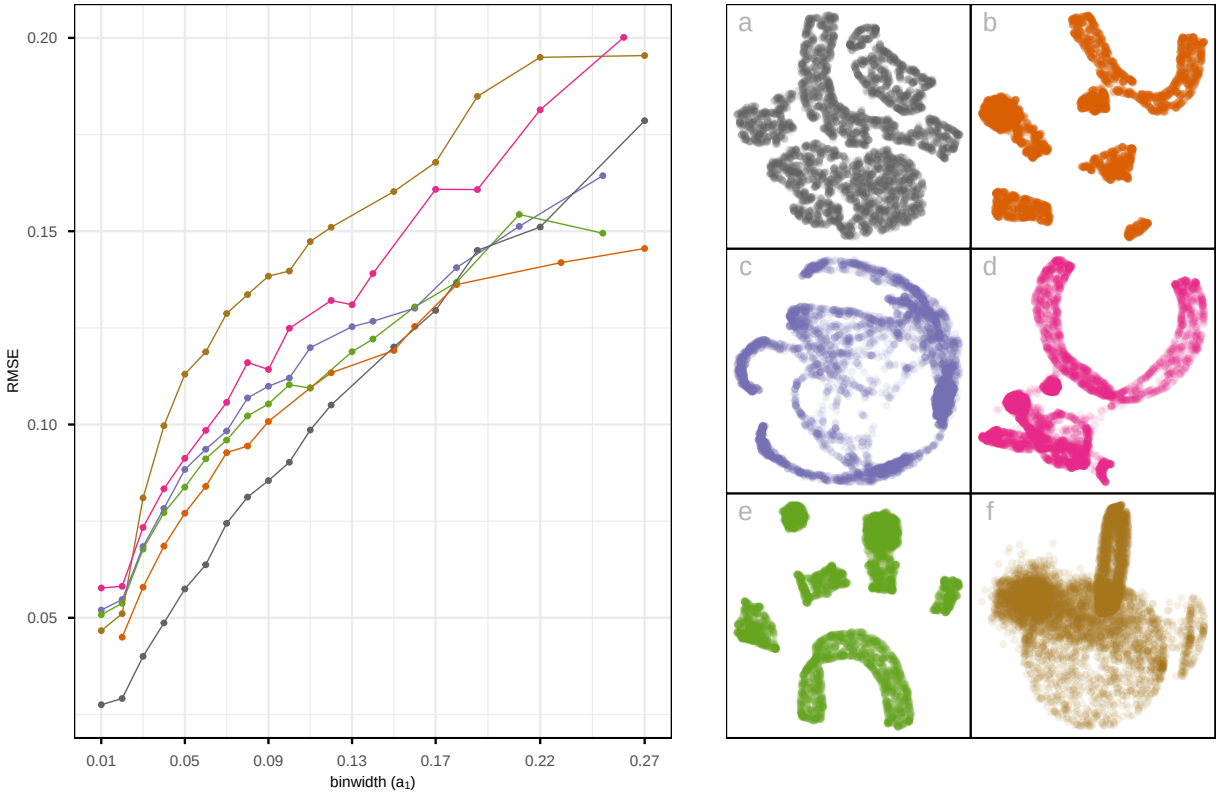

Figure 8.2: *Assessing which of the 6 NLDR layouts ((a) tSNE, (b) UMAP, (c) PAHTE, (d) TriMAP, (e) PaCMAP, and (f) PCA) of the five clusters data is the better representation using RMSE for varying binwidth (a_1). Colour is used for the lines and points in the left plot to match the scatterplots of the NLDR layouts (a-f). Layout f is universally poor. Layouts a and b are universally optimal. Layout b shows six well-separated clusters and latout a shows close clusters, thus layout a is the best choice.*

To assess their performance, we computed the root mean squared error (RMSE) between the observed high-dimensional data and the fitted values, defined as the high-dimensional mappings of the bin centroids (**?**). A lower RMSE indicates that the method better preserves the high-dimensional structure in its low-dimensional embedding.

As shown in Figure @ref(fig:fig-nldr-layouts), tSNE (Figure @ref(fig:fig-nldr-layouts) a) achieved the lowest RMSE across bin widths (mostly tiny), indicating high preservation of both local and global structures. Its layout displays well-separated clusters with minimal inter-cluster distances, making it the most faithful representation of the underlying data structure. UMAP and PaCMAP (Figure @ref(fig:fig-nldr-layouts) b and e) produced moderately accurate embeddings, although the six clusters appear more well-separated, while PHATE (Figure @ref(fig:fig-nldr-layouts) c) show non-linear cluster structures irrespective of the original structure. Also, TriMAP (Figure @ref(fig:fig-nldr-layouts) d) has high RMSE, and show three clusters with small distances. PCA (Figure @ref(fig:fig-nldr-layouts) f) failed to capture the non-linear geometry, leading to the highest RMSE.

Table 8.1: *Comparison of clustering performance metrics (within–between ratio (wb.ratio), Dunn index, Corrected Rand index, and variation of information (VI) across k-means, hierarchical, and model-based clustering methods.*

Metric	wb.ratio	Dunn Index	Corrected Rand	VI
k-means	0.61	0.01	0.42	1.32
Hierarchical	0.61	0.01	0.50	1.15
Model-based	0.61	0.01	0.75	0.65

8.3 Benchmarking clustering algorithms

To further evaluate the structure of the generated data, we benchmarked three clustering algorithms: ***k*-means** (Chapter 20 of **?**), **hierarchical** (**?**), and **model-based clustering** (**?**; **?**) using the simulated dataset. The model-based clustering was performed with the `"VVV"` covariance structure, allowing each cluster to vary in volume, shape, and orientation. Cluster validity statistics were computed using the `cluster.stats()` function from the `fpc` package (**?**).

Overall, all methods produced similar compactness and separation, as reflected by the *within–between cluster ratios (wb.ratio)* and *Dunn indices*. However, the **model-based clustering** achieved the highest *Corrected Rand Index* (0.75) and lowest *Variation of Information (VI)* (0.65), indicating the best recovery of the true underlying groups (Table @ref(tab:summaryclust-tb-pdf)). In comparison, *k*-means and hierarchical clustering showed moderate agreement with the true labels. These findings demonstrate that mixture-based approaches can more effectively capture the heterogeneity of clusters in high-dimensional, non-spherical data.

Chapter 9

Conclusion

The `cardinalR` package introduces a flexible framework for generating high-dimensional data structures with well-defined geometric properties. It addresses an important need in the evaluation of clustering, machine learning, and DR methods by enabling the construction of customized datasets with interpretable structures, noise characteristics, and clustering arrangements. In this way, `cardinalR` complements existing packages such as `geozoo`, `snedata`, and `mlbench`, while extending the scope to higher dimensions and more complex shapes.

The motivation for developing this package originated from the need to design a perception–misperception experiment, aimed at investigating how well NLDR methods preserve inter-cluster structure. To conduct this study, we required simulated datasets with carefully controlled geometric and clustering properties. While some existing packages provided useful starting points, none fully supported the creation of flexible, high-dimensional data with the specific structural variations needed for our experiment. Developing these generators for research purposes gradually led to the design of `cardinalR` as a general-purpose package, so that other researchers can benefit from the same tools for simulation, benchmarking, and teaching.

The included structures cover a wide range of diagnostic settings. Branching shapes facilitate the study of continuity and topological preservation, the Scurve with a hole allows investigation of incomplete manifolds, and clustered spheres assess separability on curved surfaces. The Möbius strip introduces challenges from non-orientable geometry, while gridded cubes and pyrholes test spatial regularity and clustering in sparse, non-convex regions.

These structures are designed to support not only algorithm diagnostics, but also teaching high-dimensional concepts, benchmarking reproducibility, and evaluating hyperparameter sensitivity. By

allowing users to adjust dimensionality, sample size, noise, and clustering properties, the package promotes transparent experimentation and comparative model evaluation.

Future extensions of `cardinalR` may include biologically inspired or application-driven data structures would further broaden its utility in domains such as bioinformatics, forensic science, and spatial analysis.

Chapter 10

Acknowledgements

The source material for this paper is available at github.com/JayaniLakshika/paper-cardinalR. This article is created using knitr (?) and rmarkdown (?) in R with the `rjtools::rjournal_article` template. These `R` packages were used for this work: `cli` (?), `tibble` (?), `gtools` (?), `dplyr` (Wickham 2023), `stats` (?), `tidyr` (?), `purrr` (?), `mvtnorm` (?), `geozoo` (?), and `MASS` (?).

Chapter 11

Development of an interactive and dynamic graphics system for NLDR users

Chapter 12

Conclusion and future plans

This thesis presents four key contributions that collectively advance the understanding and evaluation of NLDR methods. The work improves NLDR diagnostics, provides insights into human perception of high-dimensional structures, develops methods for generating clustering data structures, and implements user-friendly software tools that support exploratory analysis and visualization.

12.1 Contributions

The primary contributions of this research are fourfold. First, we introduce a novel method for visualizing how NLDR warps data, thereby improving the diagnostics of NLDR techniques. Second, we conduct a human subject experiment to investigate the perception and misperception of NLDR representations, providing evidence on how clusters at varying distances are identified in comparison to high-dimensional tours. Third, we develop two R packages: `quollr`, which implements the proposed diagnostic method, and `cardinalR`, which generates high-dimensional data structures with enhanced features such as added noise dimensions and background noise. Finally, we create a Shiny application that provides analysts with a user-friendly interface for obtaining the most accurate NLDR representation.

The software outputs of this research have been made publicly available to support transparency and reproducibility. The R package `quollr` has been on CRAN since March 2024 and has received 2950 downloads from the CRAN mirror; its development version is hosted on GitHub at https://github.com/jayanilakshika/quollr. The R package `cardinalR` has been available on CRAN since April 2024 and has received 3151 downloads from the CRAN mirror, with the latest development version at https://github.com/jayanilakshika/cardinalR. A Shiny application for `quollr` is accessible

via one of the mirror sites at https://menurar.netlify.app/, with its source code available at https://github.com/JayaniLakshika/menuraR.

The survey web application, **Match-a-roo** (https://ebsmonash.shinyapps.io/Match-a-roo/), was designed and implemented in Shiny to collect participant responses and demographic information. Each subject accessed the survey through the shinyapps.io server.

All materials associated with this thesis are openly available at https://github.com/JayaniLakshika/Monash_PhD_thesis, reflecting the principles of transparency and reproducible research. The thesis itself is written in Quarto (**?**) and published online at https://jayani-lakshika-phd-thesis.netlify.app.

In addition, a number of R packages were essential in the development of this work, including `tidyverse` (**?**), `lmtest` (**?**), `kableExtra` (**?**), `patchwork` (**?**), `glue` (**?**), `ggpcp` (**?**), `here` (**?**), and `knitr` (**?**). (Final package list to be confirmed upon completion of writing.)

12.2 Future work

There are several directions that this work can be developed.

12.2.1 Extending our algorithm to NLDR representations beyond 2-D

A potential direction for future work is extending the current algorithm to NLDR results that project into more than two dimensions. While most existing tools including those developed in this thesis focus on 2-D embeddings, exploring projections into higher dimensions like 3-D or 5-D spaces could provide richer structural information in some settings.

Binning into cubes (3-D or higher) could be performed relatively easily and used as the basis for constructing a wireframe representation of the fitted model. The algorithm for convex hull computation in p-dimensions, as described by Barber et al. (1996) and implemented in related software (Laurent (2023)), serves as inspiration for this approach. Alternatively, a simpler method using k-means clustering to obtain centroids in higher-dimensional embeddings might be feasible; however, the challenge would lie in determining how to connect these centroids to form an appropriate wireframe structure.

12.2.2 Scagnostics to evaluate NLDR

One promising direction for future work is the integration of scagnostics ((**?**), (**?**)) as an additional tool for evaluating NLDR results. Scagnostics provide a set of quantitative shape-based metrics (e.g., convexity, skewness, stringiness, clumpiness) that describe the geometric characteristics of scatterplots.

By applying these metrics to 2-*D* scatterplots generated by NLDR methods, we could obtain an objective assessment of how well these methods preserve or distort data structures, particularly in relation to their characteristics (eg: non-linearity).

Moreover, investigating how scagnostic profiles vary with different sample sizes for specific underlying data structures would provide valuable insight into the stability and robustness of NLDR methods. This could help identify which methods are more resilient to changes in sample size and which structures are more prone to distortion under small sample sizes.

12.2.3 Compare prediction approaches

Future work includes evaluating and comparing the prediction capabilities of different NLDR methods. Only some methods such as UMAP provide built-in functionality (Konopka (2023)) to project new high-dimensional observations into an existing low-dimensional embedding. Our approach introduces a general prediction framework that can be applied to any NLDR method. It works by identifying the nearest high-dimensional bin centroid for a new observation and assigning its corresponding 2-*D* centroid from the fitted model.

Having predictions from both the built-in functions (when available) and our centroid-based method allows for direct performance comparisons. This enables a systematic evaluation of how well different approaches preserve structure when projecting new observations into an existing NLDR space.

12.2.4 Interactive diagnostic tool for NLDR evaluation

A promising direction for future work is the development of an interactive tool that enables diagnostic evaluation of NLDR methods, particularly in the context of clustering. Since different NLDR techniques and parameter settings can lead to varied low-dimensional representations and possible misclassifications. It is essential to have tools that help users explore and understand the sources of these discrepancies.

We propose building a Shiny-based interactive application that allows users to upload: 2-*D* and high-dimensional Euclidean distance matrices, NLDR embeddings, and results from spin-and-brush analysis ((**?**), (**?**)).

Spin-and-brush is a dynamic visual method used to explore clustering structures in high-dimensional numerical data. It is especially helpful in identifying the influence of nuisance variables, structural differences among clusters (e.g., shape or variance), and detecting low-dimensional manifolds embedded in higher dimensions. This functionality can be implemented using the `detourr` package ((**?**)), which supports recording and replaying brushing sequences.

The envisioned tool would allow users to select a specific cluster and a data point of interest and inspect how the data point relates to its cluster through interactive 2-*D* and high-dimensional distance visualizations.

The user interface could be organized into two panels. The left panel would display the selected cluster and the specific point within the 2-*D* embedding. The right panel would show a distribution of distances from the selected point to all other points within the same cluster.

Interactive brushing between these panels would help users explore where NLDR methods preserve or distort clustering structure. This tool would not only support more intuitive diagnosis of NLDR performance but could also serve as a foundation for building automated evaluation metrics that align with human interpretation.

12.2.5 Lineup protocols to evaluate NLDR sensitivity and structure preservation

A valuable extension of this work would be to develop lineup-based evaluation protocols ((**?**)) for NLDR methods. Lineups, originally introduced as a statistical inference tool for graphical perception, involve presenting a true data plot randomly embedded among a set of null plots generated under a null model. Observers are asked to identify the plot that appears most different, allowing for an assessment of whether a visual structure stands out beyond what might be expected by chance.

Applied to NLDR, lineups could help evaluate how well a 2-*D* layout preserves the structure of the original high-dimensional data. For example, a lineup could contain one plot of the true NLDR layout and multiple null layouts generated from shuffled or noise-added versions of the data. If participants consistently identify the true layout, it suggests that the NLDR method has effectively preserved meaningful structure.

Lineups could also be extended to study the sensitivity of NLDR methods to hyperparameters. Multiple layouts could be shown, each corresponding to a different hyperparameter setting (e.g., number of neighbors in UMAP or perplexity in tSNE), to evaluate whether small parameter changes lead to perceptually different results. This would allow researchers to quantify the robustness of each method and guide more stable parameter selection.

12.2.6 Visualising experimental designs

The main objective of this tool is to visualise and validate results from experiments. It includes a web application that allows users to easily upload their experiment design data and results data for visualisation. Additionally, we plan to incorporate interactive features such as linked selections

and filters. While the tool primarily visualises categorical data, transforming continuous data into intervals can provide a useful way to visualise continuous data as well.

The initial workflow includes importing the experimental design and results, data preprocessing, 2-*D* static visualization, 2-*D* interactive visualization, and dynamic visualization. The data preprocessing steps involve mapping the design data and finding missing responses in the results, transforming the data to a wide format to compute the number of responses for each factor level combination (missing combinations are recorded as 0), and converting the data into a long format suitable for visualization. For 2-*D* static plots, `ggplot2` (**?**) is used to provide a clear view of the distribution of counts across various factor levels. `plotly` (**?**) is used to add interactivity, and hovering over the tiles reveals additional information, enhancing the user's ability to interact with and understand the data. The dynamic visualization will show each vertex as a factor level combination, with jittered points representing the number of responses for each factor combination and edges connected with one level change in a factor. Currently, the `detourr` (**?**) package is used for the implementation.

12.2.7 Investigating perception and misperception in NLDR with additional factors

While our current user study has focused on how the distance between clusters affects human perception of NLDR layouts, there remain several important factors that could further influence perception and misperception. A promising direction for future work is to systematically explore how variations in data characteristics impact a user's ability to correctly interpret dimensionality-reduced representations.

Specifically, we propose extending the perceptual study to consider:

- Background noise: Adding uniformly or normally distributed noise to the data can obscure true structure, and it is important to understand how different NLDR methods handle such interference and how users respond to it visually.
- Number of clusters: As the number of clusters increases, distinguishing them in 2-*D* may become more challenging, particularly if the separation is subtle or overlaps occur.
- Noise dimensions: Including additional high-dimensional features that contain no signal (i.e., noise variables) can affect NLDR outcomes. We aim to evaluate how this impacts perceived structure.
- Sample size: Varying the number of observations may change both the visual density and the stability of the NLDR projection, influencing the interpretability of patterns.

- Random seed: Since many NLDR methods are stochastic (e.g., tSNE, UMAP), different seeds can lead to different embeddings. It is valuable to understand whether these differences are perceptible to users and how they affect interpretability.

By extending the study to incorporate these data-driven variables, we can build a more comprehensive understanding of when and why human misperception occurs in NLDR layouts, and which methods are more resilient to such distortions. This work will support the development of more robust diagnostics and improve the practical use of NLDR.

12.2.8 Comparative perceptual study of PCA and NLDR Methods

Another valuable direction for future work is to investigate how PCA compares to NLDR methods in terms of human perception and interpretability. PCA is a linear method widely used for its simplicity and mathematical transparency, whereas NLDR methods often involve non-linear transformations and hyper-parameter tuning.

By comparing how users interpret and misinterpret PCA layouts versus NLDR generated layouts, we can gain insights into whether linear techniques are inherently easier to understand or whether they may lead to different types of visual distortions. This work would help clarify when PCA is sufficient for visual analysis and when the added complexity of NLDR is warranted, particularly for exploratory tasks that rely on visual intuition.

Bibliography

Amid, E., and Warmuth, M. K. (2019), "TriMap: Large-scale Dimensionality Reduction Using Triplets," *arXiv preprint arXiv:1910.00204*.

Barber, C. B., Dobkin, D. P., and Huhdanpaa, H. (1996), "The Quickhull Algorithm for Convex Hulls," *ACM Trans. Math. Softw.*, 22, 469--483.

Hart, C., and Wang, E. (2022), *detourr: Portable and Performant Tour Animations*.

Jolliffe, I. (2011), "Principal Component Analysis," in *International encyclopedia of statistical science*, Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 1094–1096.

Konopka, T. (2023), *umap: Uniform Manifold Approximation and Projection*.

Laa, U., Cook, D., and Lee, S. (2022), "Burning Sage: Reversing the Curse of Dimensionality in the Visualization of High-Dimensional Data," *Journal of Computational and Graphical Statistics*, 31, 40–49.

Laurent, S. (2023), *cxhull: Convex Hull*.

Lee, S., Cook, D., Silva, N. da, Laa, U., Wang, E., Spyrison, N., and Zhang, H. S. (2021), "A Review of the State-of-the-Art on Tours for Dynamic Visualization of High-Dimensional Data," *arXiv preprint arXiv:2104.08016*.

Maaten, L. V. D., and Hinton, G. E. (2008), "Visualizing Data Using t-SNE," *Journal of Machine Learning Research*, 9, 2579–2605.

McInnes, L., Healy, J., Saul, N., and Großberger, L. (2018), "UMAP: Uniform Manifold Approximation and Projection," *Journal of Open Source Software*, 3, 861.

Moon, K. R., Dijk, D. van, Wang, Z., Gigante, S. A., Burkhardt, D. B., Chen, W. S., Yim, K., Elzen, A. van den, Hirn, M. J., Coifman, R. R., Ivanova, N. B., Wolf, G., and Krishnaswamy, S. (2019), "Visualizing Structure and Transitions in High-Dimensional Biological Data," *Nature Biotechnology*, 37, 1482–1492.

Wang, Y., Huang, H., Rudin, C., and Shaposhnik, Y. (2021), "Understanding How Dimension Reduction Tools Work: An Empirical Approach to Deciphering t-SNE, UMAP, TriMap, and PaCMAP for Data Visualization," *Journal of Machine Learning Research*, 22, 1–73.

Wickham, H. (2023), *conflicted: An Alternative Conflict Resolution Strategy*.

Wickham, H., Cook, D., Hofmann, H., and Buja, A. (2011), "tourr: An R Package For Exploring Multivariate Data With Projections," *Journal of Statistical Software*, 40, 1–18.

Appendix A

Appendix to “Visualising how non-linear dimension reduction warps your data”

Appendix B

Appendix to “A user study to explore perception and misperception in NLDR representations”

Appendix C

Appendix to “quollr”

Appendix D

Appendix to “cardinalR”

Appendix E

Appendix to “Development of an interactive and dynamic graphics system for NLDR users”